Angewandte KI & KI-Engineering (AKE) · Block C – KI-Engineering & MLOps

Daten-Engineering für ML

Vorlesung · Woche 6

Stefan Gnaase · PHWT · Wintersemester 2026/27

Nur zur Verwendung im Kurs Angewandte KI & KI-Engineering (AKE). Weitergabe nicht gestattet.

Organisatorisches – Woche 6

Heute: Vorlesung mit Gruppenvortrag und drei Lab-Einschüben, am Ende Capstone-Kickoff

Termin	KW 46 (12.11.) · 270 Min.: Vorlesung, Gruppenvortrag, Lab-Einschübe
Vorher	Kursordner aktualisieren: git pull · uv sync – DVC 3.67.1 ist schon in der Umgebung
Gruppe	Gruppenvortrag „Datenqualität“ · 80 Min. nach Abschnitt 3
Capstone	Heute Kickoff Datenkonzept · ab heute 2-SWS-Block = Projektarbeitszeit mit Austausch
Online	doc.dvc.org · docs.feast.dev · Huyen (2022) Kap. 3–4 im Lektüreordner

Roter Faden und Ablauf heute

Block C beginnt: Das Trägermodell bekommt eine versionierte, reproduzierbare Datenbasis

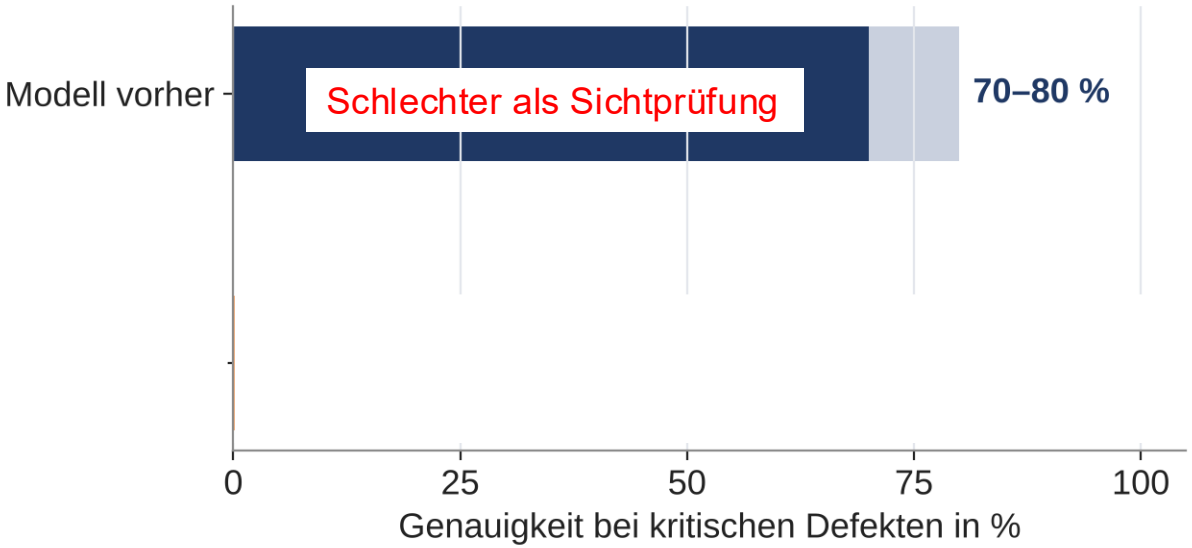
Modelle bauen → **Modelle produktiv machen** → **Modelle verantworten** → **Capstone**

heute: Start von Block C – das Trägermodell wandert jetzt durch den Lifecycle

#	Abschnitt	Leitfrage	Lab
1	Wozu Daten-Engineering?	Warum ist das Modell der kleinste Teil?	–
2	Datenquellen & -formate	Wie liegen Daten vor – und warum ist das wichtig?	–
3	Datenpipelines	Wie wird aus einem Notebook eine Pipeline?	–
G	Gruppenvortrag	Datenqualität: Validierung · Label-Noise · Imbalance · Leakage	–
4	Datenversionierung mit DVC	Welche Daten stecken in welchem Modell?	L1, L2
5	Feature Stores	Wie bekommt das Modell im Betrieb dieselben Merkmale?	–
6	Bias in Trainingsdaten	Bildet der Datensatz die Einsatzwelt ab?	L3
C	Capstone-Kickoff	Woher kommen Ihre Daten – und dürfen Sie sie nutzen?	L4

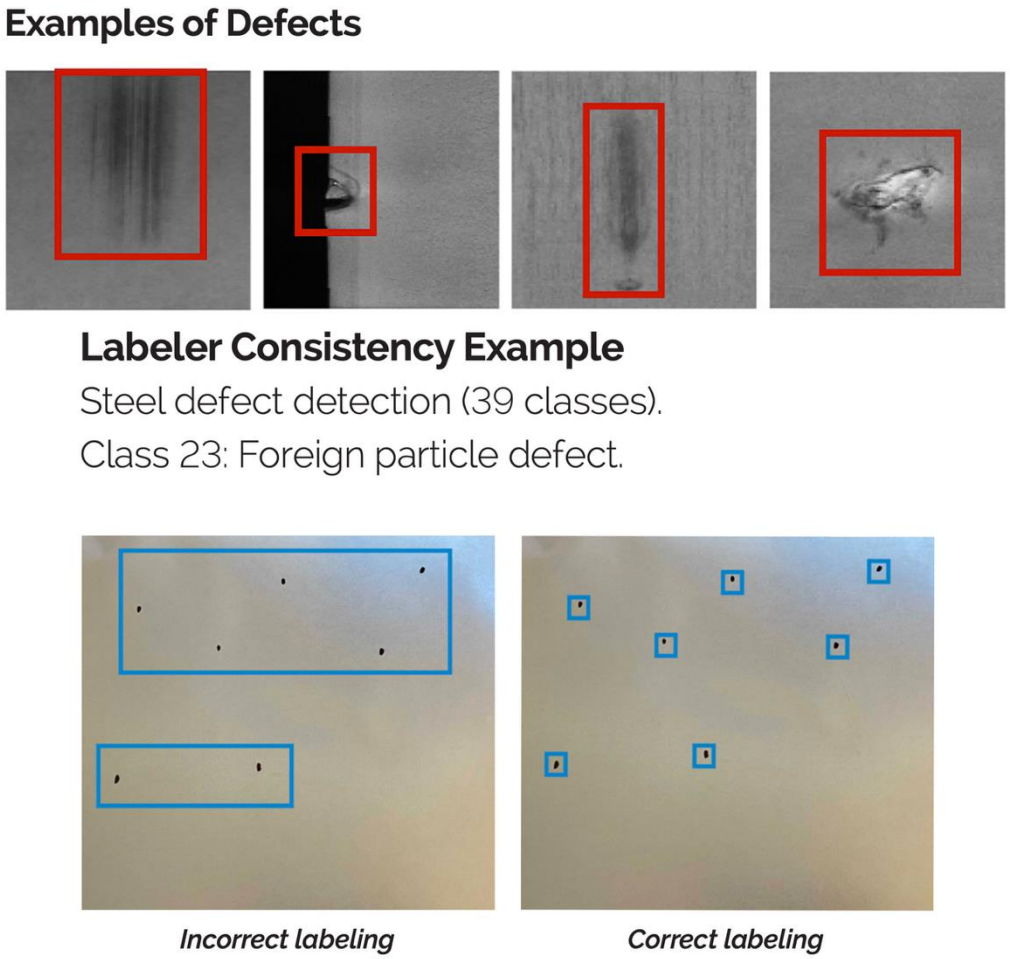
Wozu brauchen wir das?

Sichtprüfung in der Stahlproduktion: Nicht Modell, sondern Daten waren das Problem



< 10 %

der Projektzeit für Modelliterationen
Restzeit für Daten- und Labelarbeit



→ Unser Leitbeispiel heute: das **Trägermodell** ➔ es bekommt eine versionierte, reproduzierbare Datenbasis

Quellen: Landing AI, Case Study Steel Inspection (Herstellerangabe, ohne Peer Review) · Sambasivan et al. (2021), CHI '21 · Foto: https://landing.ai/wp-content/uploads/2021/07/LandingAI_CaseStudy_Steel_Inspection.pdf

Die Geschichte des Tages

Erst verstehen, wie Daten vorliegen und fließen, dann versionieren, dann auf Betrieb und Bias schauen



1 Wozu

Was macht Data-Engineering, warum kleinster Teil?

Datenabhängigkeiten sind die teuerste Schuld



2 Formate

Wie liegen Daten vor?

Zeilen schreiben, Spalten lesen



3 Pipelines

Wie wird aus dem Notebook eine Pipeline?

Stufen mit festen Ein- und Ausgaben



G Datenqualität

Gruppenvortrag

Validierung · Label-Noise · Imbalance · Leakage



4 DVC

Welche Daten stecken in welchem Modell?

Git versioniert Zeiger, DVC die Daten



5 Feature Stores

Dieselben Merkmale im Betrieb?

Point-in-time statt Blick in die Zukunft



6 Bias

Bildet der Datensatz die Einsatzwelt ab?

Fallschirm = blauer Himmel?



7 Capstone

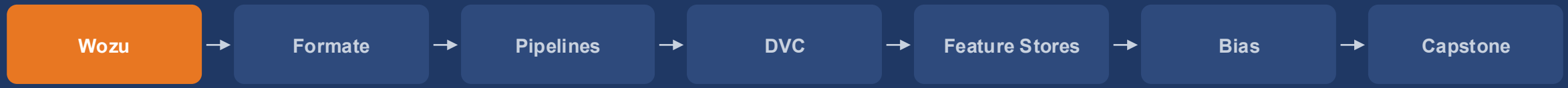
Woher kommen Ihre Daten?

Datenkonzept auf einer Seite

1

Wozu Daten-Engineering?

Leitfrage Warum ist das Modell der kleinste Teil?



Was Daten-Engineering konkret macht

Fiktives Beispiel Kratzererkennung an Blechteilen: vom Kamerabild bis zum Trainingsdatensatz

1 Erfassen

Die Kamera an Presse 3 fotografiert jedes Teil: rund 2.000 Bilder pro Schicht.

RISIKO OHNE PLAN
Wo liegen die Bilder und fehlen welche?

2 Labeln

Eine Prüferin markiert jedes Bild als „Kratzer“ oder „OK“.

RISIKO OHNE PLAN
Ist ein 1-mm-Kratzer ein Fehler? Jeder entscheidet anders.

3 Prüfen

Unscharfe, doppelte und leere Bilder werden aussortiert.

RISIKO OHNE PLAN
Wer merkt, dass die Linse verschmutzt ist?

4 Aufbereiten

Zuschneiden, auf 224 px skalieren, in Training und Validierung teilen.

RISIKO OHNE PLAN
Ist der Split beim nächsten Mal derselbe?

5 Bereitstellen

Der geprüfte Datensatz „v3“ geht ins Training.

RISIKO OHNE PLAN
Welche Datenversion steckt im Modell an der Linie?

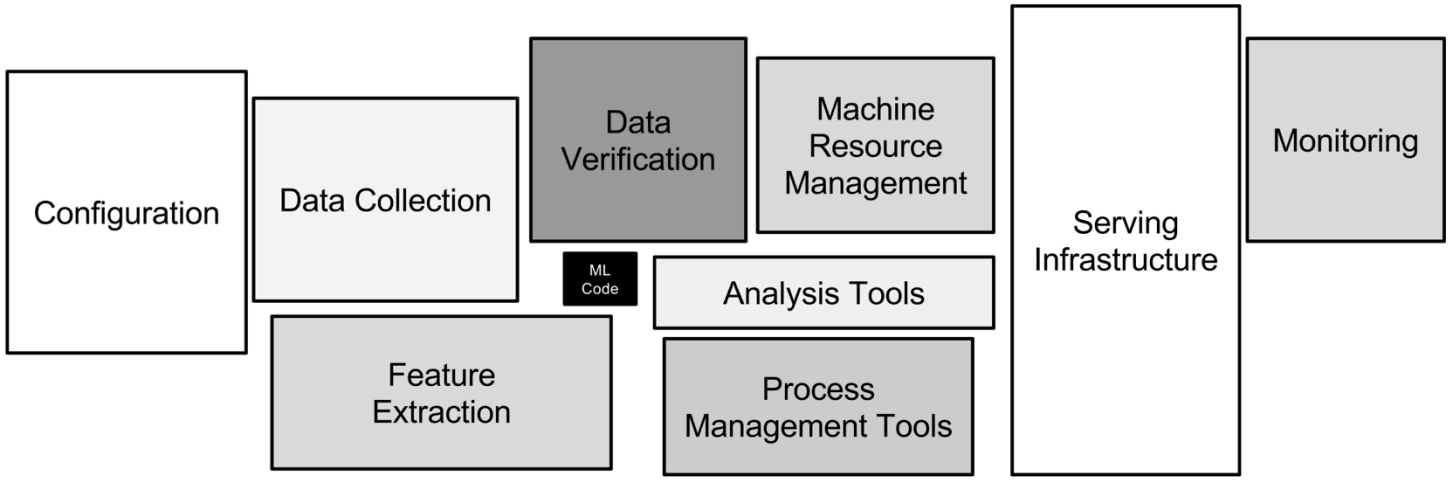
Daten-Engineering sorgt dafür, dass Daten zuverlässig, geprüft und nachvollziehbar beim Modell ankommen. Und das bei jedem Training gleich.

Analogie Fertigung: Wareneingangsprüfung + Lager + Chargenrückverfolgung – nur für Daten statt für Bauteile

→ Data Science fragt: **Welches Modell?** Daten-Engineering fragt: **Welche Daten und bekomme ich sie wieder genauso?**

Nur ein kleiner Kasten ist Modellcode

Ein ML-System besteht vor allem aus der Infrastruktur um das Modell herum



Heute im Fokus

Datenerhebung

Datenprüfung

Merkmalsextraktion

ML-Code
der kleinste Kasten in der Mitte

außerdem: Konfiguration · Ressourcenverwaltung · Analysewerkzeuge · Prozessverwaltung · Serving-Infrastruktur · Monitoring

Abb.: Sculley, D. et al.: Hidden Technical Debt in Machine Learning Systems, 2015, NeurIPS 28, S. 4, Abb. 1

Code bricht laut, Daten brechen leise

Datenabhängigkeit: Ihr Modell nutzt Daten, die ein anderes System erzeugt – und jederzeit ändern kann

Codeabhängigkeit

Ihr Skript braucht die Bibliothek pandas. Nach einem Update fehlt sie.

```
import pandas as pd
ModuleNotFoundError: No module named 'pandas'
```

FOLGE

Programm stoppt sofort mit Fehlermeldung. Sie bemerken das Problem in Sekunden.

Datenabhängigkeit

Ihr Modell nutzt die Spalte `temp` aus dem MES. Die IT stellt die Einheit von °C auf °F um.

Messung	vorher (°C)	nachher (°F)
08:00	62,0	**143,6**
08:01	61,5	**142,7**
08:02	63,1	**145,6**

FOLGE

Keine Fehlermeldung. Das Modell rechnet weiter und liegt falsch, bis jemand die steigende Ausschussquote bemerkt.

→ Deshalb: **Daten brauchen dieselbe Sorgfalt wie Code ➔** prüfen, versionieren, Änderungen nachvollziehen

Datenabhängigkeiten kosten mehr als Code

Codeabhängigkeiten prüft der Compiler | Datenabhängigkeiten prüft niemand



1 Instabil

Ein Signal aus einem anderen System ändert sich – das Modell merkt es nicht

BEISPIEL

Sensor misst 2 °C zu viel – Modell lernt „72 °C = normal“. Nach Neukalibrierung wirken echte 70 °C plötzlich zu kalt

GEGENMITTEL

Datenstand v1 einfrieren – v2 erst nach Retraining und Vergleich übernehmen



2 Ungenutzt

Merkmale mit kaum Nutzen machen das System verletzlich

BEISPIEL

alte und neue Artikelnummern parallel – dann fällt die alte weg

GEGENMITTEL

regelmäßig ein Merkmal weglassen und testen



3 Pipeline-Dschungel

Scrapes, Joins, Sampling-Schritte, Zwischendateien

BEISPIEL

jede Änderung braucht einen teuren End-to-End-Test

GEGENMITTEL

Stufen mit festen Ein- und Ausgaben

→ Gegenmittel heute: **versionierte Daten** (DVC) und eine **deklarierte Pipeline** (dvc.yaml)

Unser Trägermodell heute

Nach Lab 2 gibt es ein gutes Modell aber keinen Nachweis, aus welchen Daten es entstand


Kettensäge


Kirche


Kassettenrecorder



Golfball


Fallschirm

data/imagenette5/
3.000 Bilder · 28,5 MB · in Git committed

lab02_loesung.ipynb
Reihenfolge = Klickfolge

models/traeger_v1.*
Val-Genauigkeit 98,4 % · .pt nur lokal (gitignored)

Ein Bild wird ersetzt. Welches Modell gehört dann zu welchen Daten?

Zellen in anderer Reihenfolge. Kommt dasselbe Modell heraus?

Die .pt liegt nur bei Ihnen. Wie kommt das Team daran?

→ Ziel heute: **Daten, Code und Modell gemeinsam versioniert ➔** ein Befehl baut alles nach

Bilder: Imagenette (J. Howard, fast.ai) / ImageNet – nur nicht-kommerzielle Forschung und Lehre · Kursrepo AKE26, 03.11.2026

Zwischenstand: Wozu Daten-Engineering?

1

Modellcode ist der kleinste Kasten

Daten, Prüfung, Merkmale, Serving, Monitoring liegen drumherum

2

Datenabhängigkeiten sind unsichtbar

instabil · ungenutzt · Pipeline-Dschungel

3

Das Trägermodell ist noch nicht reproduzierbar

Daten in Git, Modell nur lokal, Reihenfolge im Notebook

?

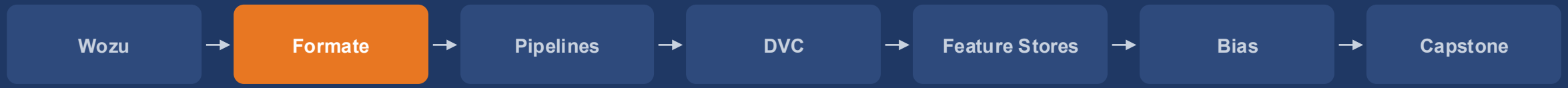
Welche Daten fließen in eine Qualitätsprognose Ihrer Fertigungslinie und wer ändert sie?

Weiter: Bevor wir versionieren, schauen wir, wie Daten vorliegen → Datenquellen & -formate

2

Datenquellen & -formate

Leitfrage Wie liegen Daten vor und warum ist das wichtig?



Datenquellen

Woher Daten kommen, bestimmt, wie viel Prüfung sie brauchen



Prüfstärke **hoch**

1 Nutzereingaben

Text, Bilder, Formulare · oft fehlerhaft · schnell verarbeiten

Fertigung: Werkerrückmeldung am Terminal, Foto vom Fehlerbild

Prüfen: Format, Plausibilität, Vollständigkeit

Prüfstärke **mittel**

2 Systemdaten und Logs

Logs, Modellausgaben, Verhalten · wächst schnell

Fertigung: Steuerung, Sensor- und Alarmlogs

Prüfen: Einheiten, Zeitzonen, Sensortausch

Prüfstärke **gering**

3 Interne Datenbanken

Bestand, Kunden, Aufträge · von Anwendungen gepflegt

Fertigung: ERP und MES: Aufträge, Chargen, Prüfergebnisse

Prüfen: Bedeutungsänderungen von Feldern

Prüfstärke **hoch**

4 Daten Dritter

First-, Second-, Third-Party · gekauft, vorverarbeitet

Fertigung: Wetterdaten, Lieferantendaten

Prüfen: Herkunft, Lizenz, Vorverarbeitung

→ Trägermodell: **Bilder + Dateilisten + Klassenliste** – eine kuratierte Fremdquelle (ImageNet-Teilmenge)

Einteilung nach Huyen (2022), Kap. 3 · Prüfstärke: eigene Einordnung · Illustration: eigene Darstellung (KI-gestützt)

Zeilen- oder spaltenorientiert

Dieselbe Tabelle, andere Reihenfolge im Speicher – bei Sensordaten macht das Faktor 100

Zeit	Maschine	Temp.
08:00	M1	41.2
08:01	M2	39.8
08:02	M1	41.5
08:03	M2	40.1

eine Tabelle, vier Messungen

CSV · zeilenorientiert

08:00	M1	41.2	08:01	M2	39.8	08:02	M1	41.5	08:03	M2	40.1

neue Zeile = ans Ende schreiben · eine Spalte lesen = über alle Zeilen springen

Parquet · spaltenorientiert

08:00	08:01	08:02	08:03	M1	M2	M1	M2	41.2	39.8	41.5	40.1

Temperatur lesen = ein zusammenhängender Block (Column Chunk)

GUT FÜR …

Zeilenorientiert

wenn ständig neue Datensätze angehängt werden – z. B. jede Sekunde eine Messung

→ **CSV, JSON, Datenbankzeilen**

GUT FÜR …

Spaltenorientiert

wenn wenige Spalten aus sehr vielen Zeilen gelesen werden – z. B. 3 von 200 Kanälen

→ **Parquet**

RECHENBEISPIEL FERTIGUNG

Ein Jahr Sensordaten, 1 Messung pro Sekunde, 200 Kanäle. Ausgewertet werden 3 Kanäle.

365 × 24 × 3 600 s ≈ **31,5 Mio. Zeilen**
31,5 Mio. Zeilen × 200 Kanäle ≈ **6,3 Mrd. Werte**

CSV
alle 6,3 Mrd. Werte lesen **≈ 38 GB**

6,3 Mrd. Werte × 6 Byte (Text) ≈ **37,8 GB**

Parquet
nur 3 Spalten lesen **≈ 0,38 GB**

3 × 31,5 Mio. ≈ 95 Mio. Werte × 4 Byte ≈ **378 MB**

→ rund 100× weniger zu lesen

37,8 GB ÷ 0,378 GB = **100** = 200/3 (Spalten) × 6/4 (binär)
Text ≈ 6 Byte je Wert („41.25,“), float32 = 4 Byte

→ Für Ihr Capstone: **Rohdaten zeilenweise sammeln**, für Analyse und Training **als Parquet ablegen**

Prinzip: Huyen (2022), Kap. 3, S. 54–57 · Rechenbeispiel: eigene Konstruktion (ohne Kompression)

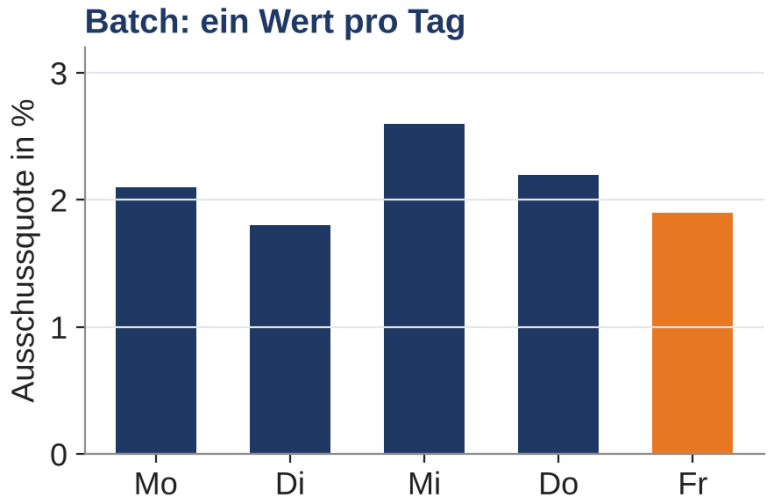
Batch oder Stream

Historische Daten in Ruhe verarbeiten / oder Ereignisse, während sie eintreffen

Batch

Datenbank · Data Lake · periodischer Job (z. B. täglich) · statische Merkmale

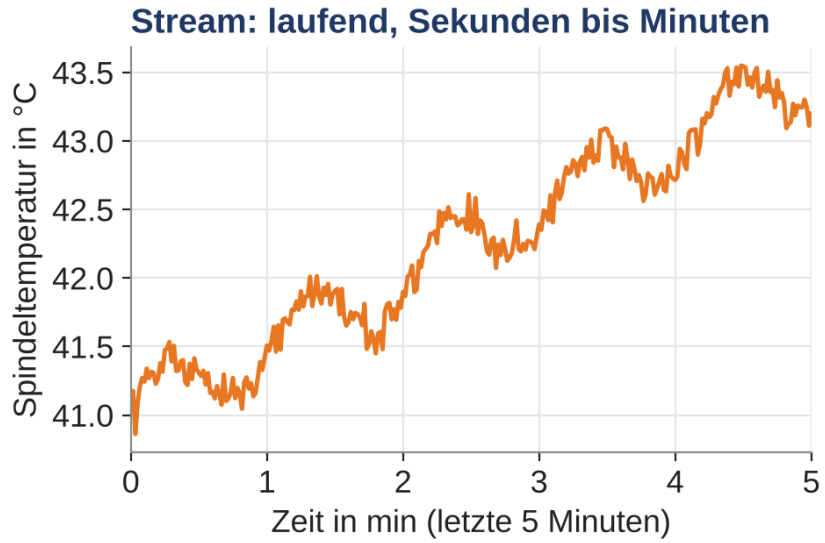
Fertigung: Ausschussquote der Linie gestern



Stream

Ereignisstrom (Kafka, Kinesis) · laufend, Sekunden bis Minuten · dynamische Merkmale

Fertigung: Spindeltemperatur der letzten 5 Min.



→ Viele Systeme brauchen **beides**, und genau dort entstehen Unterschiede zwischen Training und Betrieb (→ Feature Stores)

ETL oder ELT

Erst umformen, dann laden / oder erst alles ablegen und später umformen

ETL

+ sauber, abfragebereit
− Schemaänderungen teuer



ELT

+ schnelle Ankunft, flexible
− Suche im Rohdatenberg



→ Trägermodell = ein Mini-ETL: **Bilder prüfen** → **Dateiliste schreiben** → **in die Pipeline laden** (Stufe prepare)

nach Huyen (2022), Kap. 3, Abb. 3-7

Zwischenstand: Datenquellen & -formate

1

Die Quelle bestimmt die Prüfung

Nutzereingaben streng prüfen · Logs wachsen schnell

2

Zeilen schreiben, Spalten lesen

CSV/JSON zum Anhängen, Parquet zum Auswerten · 3 von 200 Kanälen: 38 GB vs. 0,38 GB

3

Batch und Stream, ETL und ELT

wann gerechnet und wo umgeformt wird

?

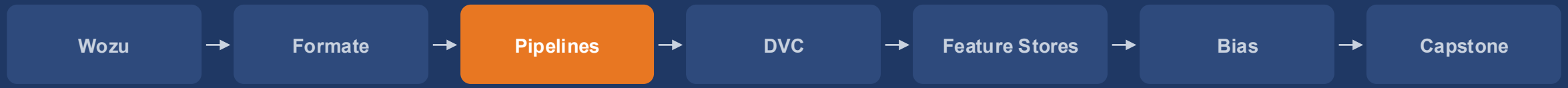
Ein Jahr Sensordaten, 200 Kanäle, ausgewertet wird kanalweise: welches Format?

Weiter: Aus einzelnen Schritten wird ein Ablauf → Datenpipelines

3

Datenpipelines

Leitfrage Wie wird aus einem Notebook eine Pipeline?



Das allgemeine Schema einer ML-Pipeline

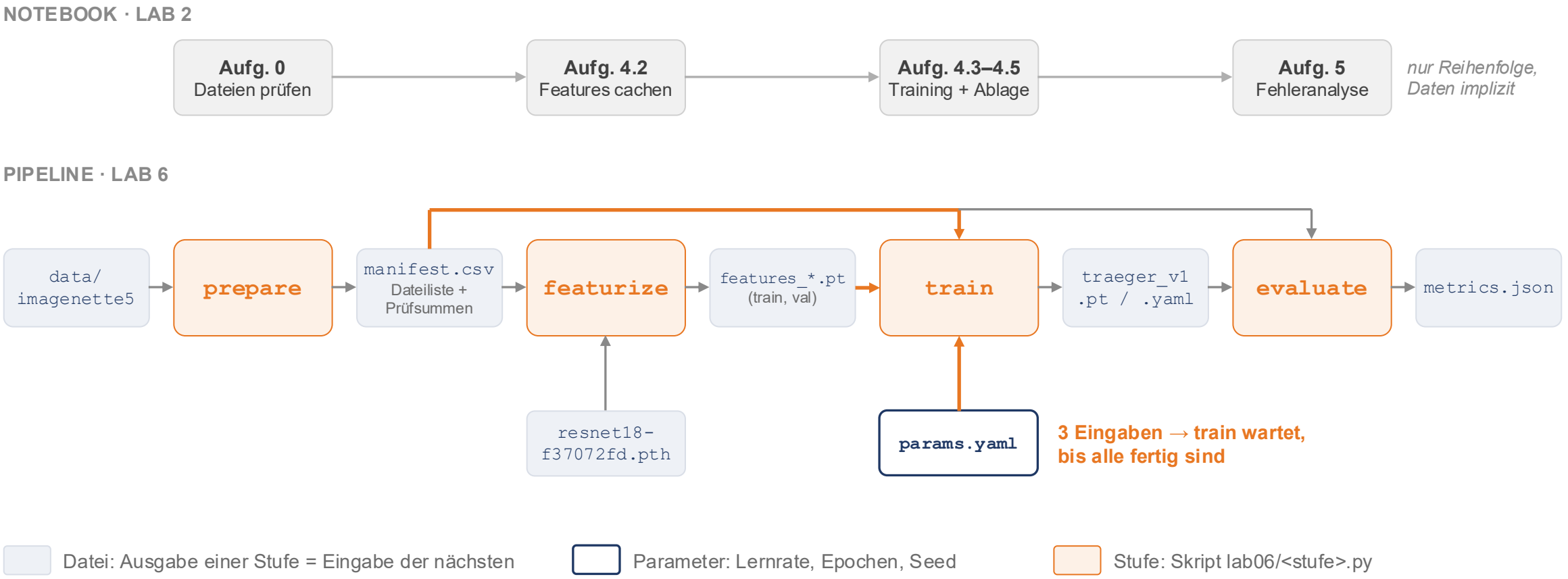
Jede ML-Pipeline durchläuft dieselben Phasen in derselben Reihenfolge



→ Allgemein gilt die **Reihenfolge der Phasen** ➔ wie sie in Stufen geschnitten werden, hängt vom Modell ab

Vom Notebook zur Pipeline

Vier Skripte mit deklarierten Ein- und Ausgaben statt einer Kette von Notebook-Zellen



→ DAG = **gerichtet, ohne Zyklen**: Eine Stufe läuft erst, wenn alle ihre Eingaben fertig sind

Reproduzierbar und gecacht

Gleiche Eingaben ergeben gleiche Ausgaben ➔ was sich nicht geändert hat, wird nicht neu gerechnet

REPRODUZIERBAR HEISST

Eingaben deklariert
Daten, Code, Parameter

Zufall fest
torch.manual_seed(0)

Umgebung fest
uv.lock

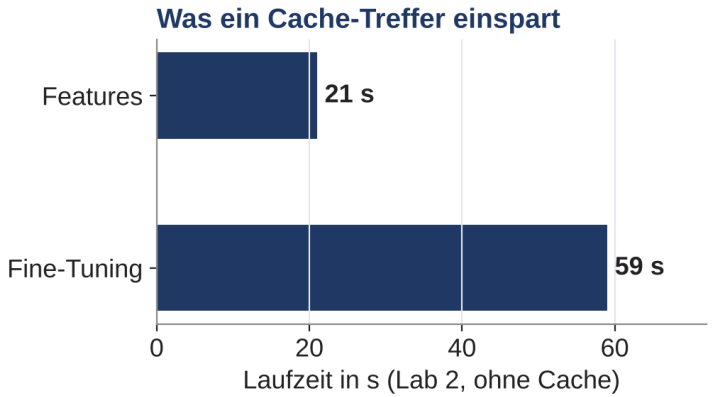
Ausgaben an festem Ort
models/traeger_v1.*

CACHING JE STUFE

Eingaben einer Stufe → Fingerabdruck (Hash)

schon gerechnet? ja: überspringen

nein: ausführen



→ Lab 2 hatte schon einen Cache **von Hand programmiert**. Eine Pipeline macht das für jede Stufe automatisch

Lab 2, Aufg. 4.2: cached_features() prüfte weights_sha und Bildanzahl · Laufzeiten: Musterlösung Lab 2

Wo entstehen Qualitätsprobleme?

Jede Stufe kann Fehler einschleppen



1

Validierung

fehlende, defekte, doppelte Dateien

→ Gruppenvortrag

2

Label-Noise

falsche oder widersprüchliche Labels

→ Gruppenvortrag

3

Class Imbalance

seltene Klassen, schiefe Splits

→ Gruppenvortrag

4

Data Leakage

Zukunft oder Testdaten im Training

→ Gruppenvortrag

→ Gleich: **Gruppenvortrag Datenqualität**
danach: wie wir Prüfungen und Korrekturen versionieren

Einordnung nach Huyen (2022), Kap. 4, S. 81–82 · Themen laut Zuschnitt des Gruppenvortrags

G

Gruppenvortrag: Datenqualität

Leitfrage Wie erkennt und behandelt man schlechte Trainingsdaten?

Gruppe	Lennart Klatte, Kacper Kostanski, Jan Wulfers, Jonas Heiker
Zeit	80 Min. · Fragen am Ende
Themen	Validierung · Label-Noise · Class Imbalance · Data Leakage

Danach: Wie wir Prüfungen und Korrekturen mit DVC versionieren

Zwischenstand: Pipeline und Qualität

1

Prüfungen werden eine Pipeline-Stufe

prepare prüft Dateien, Formate, Klassen ➔ bei jedem Lauf, nicht einmal von Hand

2

Jede Korrektur ist eine neue Datenversion

Label korrigiert → neues Modell: Welche Version war es?

3

Splits gehören zu den Daten

train.txt und val.txt werden mitversioniert ➔ sonst droht Leakage

Weiter: Versionen brauchen ein Werkzeug → Datenversionierung mit DVC

4

Datenversionierung mit DVC

Leitfrage Welche Daten stecken in welchem Modell?



Ein Modell, viele Zutaten

Das Ergebnis hängt von Daten, Vorverarbeitung, Code und Parametern ab ➔ dieser Zusammenhang muss festgehalten werden

ZUTATEN DES TRÄGERMODELLS

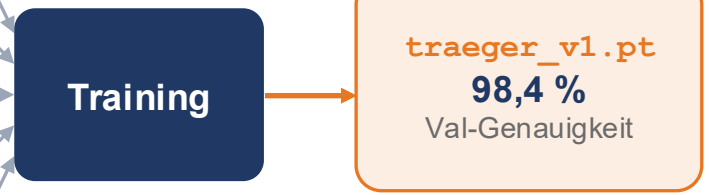
Daten
`data/imagenette5/` · 3.000 Bilder

Vorverarbeitung
Crop 160 px · Normalisierung · Augmentierung

Vortrainierte Gewichte
`resnet18-f37072fd.pth`

Code
`traeger.py` · `lab06/*.py`

Parameter
`params.yaml` · Lernrate, Epochen, Seed



ERGEBNIS

`traeger_v1.pt`
98,4 %
Val-Genauigkeit

Eine Zutat ändern → anderes Modell
„Changing Anything Changes Everything“
Sculley et al. 2015

WAS DVC FÜR JEDES MODELL LEISTET

1 **Festhalten**
Welche Version jeder Zutat steckt in diesem Modell?
`dvc.lock`

2 **Feststellen**
Was hat sich seit dem letzten Lauf geändert?
`dvc status` · `dvc params diff`

3 **Wiederherstellen**
Wie komme ich zurück zum Stand von letzter Woche?
`git checkout` + `dvc checkout`

→ **Ergebnis = f(Daten, Vorverarbeitung, Code, Parameter)** ➔ DVC hält fest, welche Version jeder Zutat in welchem Modell steckt

Git und DVC

Git versioniert kleine Textdateien · DVC legt die großen Daten daneben, Git merkt sich nur den Zeiger

Arbeitsordner

`data/imagenette5/`
3.000 Bilder · 28,5 MB

Git ignoriert den Ordner (data/.gitignore)

Warum nicht einfach ins Git?
GitHub lehnt Dateien > 100 MB ab, und Binärdaten lassen sich nicht sinnvoll vergleichen.

`dvc add`

GIT · Code und Zeiger

`data/imagenette5.dvc`

```
outs:
- md5: fb66…e7.dir
  nfiles: 3003
  path: imagenette5
```

Zeiger statt Daten: 114 Byte, kommt ganz normal ins Git (git add, git commit)

DVC · die eigentlichen Daten

`.dvc/cache (lokal)`

Kopie der Bilder, benannt nach md5

Liegt zunächst nur auf diesem Rechner – wie die Daten ins Team kommen, zeigen git push und dvc push

→ **Git** für alles, was man liest und vergleicht · **DVC** für alles, was groß oder binär ist

Logos: Git (Jason Long, CC BY 3.0), DVC (Iterative) über Simple Icons (CC0) – Marken der jeweiligen Inhaber

Vom Inhalt zum Ablageort

DVC benennt jede Datei nach der MD5-Prüfsumme ihres Inhalts ➔ daraus ergibt sich ihr Platz im Cache

Datei



ILSVRC2012_val_00008116.JPEG
Kirche · 9 583 Byte

md5 →

MD5-Prüfsumme des Inhalts · 32 Hexadezimalzeichen

`ec766f1bb7dda82fcdff1315fc7b055c`

2 Zeichen → Ordner

30 Zeichen → Dateiname

Pfad im Cache

`.dvc/cache/files/md5/ec/766f1bb7dda82fcdff1315fc7b055c`

Ein Byte anders – anderer Hash

`"a\n"` → 60b725f1… `"a"` → 0cc175b9…

Der Cache bleibt flach

256 Ordner 00 … ff – bei 3.004 Dateien im Mittel knapp 12 pro Ordner

Der Name spielt keine Rolle

Dateien mit gleichem Inhalt liegen nur einmal im Cache

→ **Adresse = Prüfsumme des Inhalts** – nicht der Dateiname (inhaltsadressierter Speicher)

Bilder: Imagenette (J. Howard, fast.ai) / ImageNet – nur nicht-kommerzielle Forschung und Lehre

Beispiel: Gleicher Inhalt, gleicher Pfad

Vier Einträge, ein Cache. Wohin legt DVC was?

Datei	MD5 (gekürzt)	Ablage unter files/md5/
classes.json	`cf`680fa6…debb66b	`cf`/680fa6…
a.txt (echo "a")	`60`b725f1…8191b3	`60`/b725f1…
kopie.txt (cp a.txt)	`60`b725f1…8191b3	**schon da – nichts kopiert**
data/imagenette5/	`fb`66058b…5403e7.dir	`fb`/66058b….dir

Regel: Pfad = files/md5/ + 2 Zeichen + / + 30 Zeichen

Ordner: Der Hash gehört zu seiner Inhaltsliste, der .dir-Datei – nächste Folie

Ausschnitt aus dem Cache

```
.dvc/cache/files/md5/
├─ 60/
│   └─ b725f1…8191b3        a.txt, kopie.txt
├─ cf/
│   └─ 680fa6…debb66b       classes.json
├─ fb/
│   └─ 66058b…5403e7.dir    imagenette5/
└─ … (256 Ordner)
```

→ **Gleicher Inhalt = gleicher Pfad** – egal, wie die Datei heißt

Hashes: md5sum und DVC 3.67.1, Kursdatensatz data/imagenette5, 09.11.2026

Ein Ordner wird zur Liste: die .dir-Datei

Jedes Glied findet das nächste über einen Hash ➔ vom Git-Commit bis zum Bild im Arbeitsordner

Git

imagenette5.dvc

```
outs:
- md5: fb66058b…
    …5403e7.dir
  size: 28531866
  nfiles: 3003
  path: imagenette5
```

114 Byte – wird committet

lokaler DVC-Cache

.dir-Datei

`fb/66058b….dir`

```
[{"md5": "63a96dc9…",
  "relpath": "…/9404.JPEG"},
 {"md5": "ec766f1b…",
  "relpath": "…/8116.JPEG"},
 … ]
```

3.003 Einträge · 286.675 Byte

Bilddateien



`63/a96dc9…`



`ec/766f1b…`

benannt nach Hash, ohne Ordnerstruktur

Arbeitsordner

data/imagenette5/

```
train/
├─ n02979186/
│  └─ …9404.JPEG
├─ n03028079/
│  └─ …8116.JPEG
└─ …
```

3.003 Dateien · 28,5 MB

→ **Git sichert nur das erste Glied ➔** .dir-Liste und Bilder liegen zunächst nur im lokalen Cache

Werte: eigener Lauf mit DVC 3.67.1 am Kursdatensatz data/imagenette5, 09.11.2026

Selbst nachvollziehen

Wenden Sie die Regel auf einen neuen Fall an

SITUATION

`train/n03888257/`



Fallschirm-Klasse, Originale

100 ×

kopieren, neue Namen

`train_kopie/`

```
bild_001.JPEG
bild_002.JPEG
…
bild_100.JPEG
```

```
$ dvc add data/imagenette5
```

data/imagenette5 ist schon mit DVC versioniert (Stand v1)

Selbst nachvollziehen

a Wie viele neue Bilddateien landen im Cache?

b Was ändert sich in imagenette5.dvc?

c Wie lautet der Cache-Pfad zur Prüfsumme `02c34d64bf2b98fe28e23ba46a667b3b`

→ **Tipp:** Erst fragen, was sich am Inhalt ändert ➔ dann, was sich an der Liste ändert

Bilder: Imagenette (J. Howard, fast.ai) / ImageNet – nur nicht-kommerzielle Forschung und Lehre

Lösung: nachgemessen mit DVC 3.67.1

Die 100 Kopien kosten im Cache fast nichts, neu ist nur die Inhaltsliste

a **0** **neue Bilddateien im Cache**

Cache: 3.004 → 3.005 Dateien – die neue ist die .dir-Liste (295.175 Byte)

b **imagenette5.dvc** – git diff

```
 outs:
-- md5: fb66058b5aead6fc0693e0145f5403e7.dir
-   size: 28531866
-   nfiles: 3003
+- md5: 069cf1f81ba53f7f7fe9119c3658ad96.dir
+   size: 29205032
+   nfiles: 3103
    hash: md5
    path: imagenette5
```

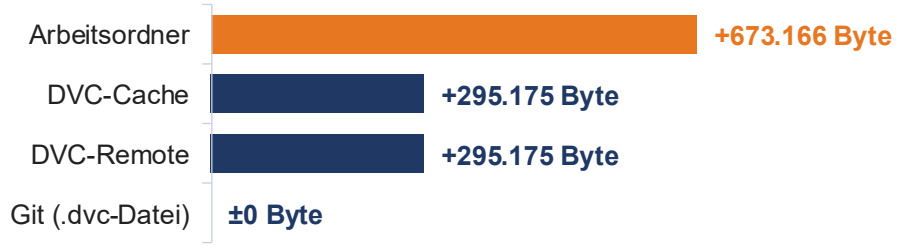
c **Cache-Pfad**

```
.dvc/cache/files/md5/02/
c34d64bf2b98fe28e23ba46a667b3b
```



Dahinter steckt dieses Bild:
`train_kopie/bild_001.JPEG`
= Original
`train/n03888257/…2508.JPEG`

Was wächst wirklich? Zuwachs durch die 100 Kopien



→ **Neue Version = neue Liste ➔** unveränderte Inhalte werden nicht noch einmal gespeichert

Werte: DVC 3.67.1, Linux/ext4, 09.11.2026 · Bild: Imagenette (fast.ai) / ImageNet, nur Forschung und Lehre

Wofür DVC den Hash braucht

Vier Befehle, ein Prinzip: Hashes vergleichen statt Dateien vergleichen

`dvc status` — Hat sich etwas geändert?

```
data/imagenette5.dvc:
  changed outs:
    modified: data/imagenette5
```

vergleicht die Hashes im Ordner mit der .dvc-Datei

`dvc push` — Was fehlt auf dem Remote?

```
3004 files pushed     erster Push
1 file pushed         nach den Kopien
```

lädt nur Hashes hoch, die dort noch fehlen

`dvc pull` — Was fehlt lokal?

```
A       data/imagenette5/
```

holt fehlende Hashes, baut dann alle Dateien auf

`dvc checkout` — Welche Version gehört hierher?

```
M       data/imagenette5/
```

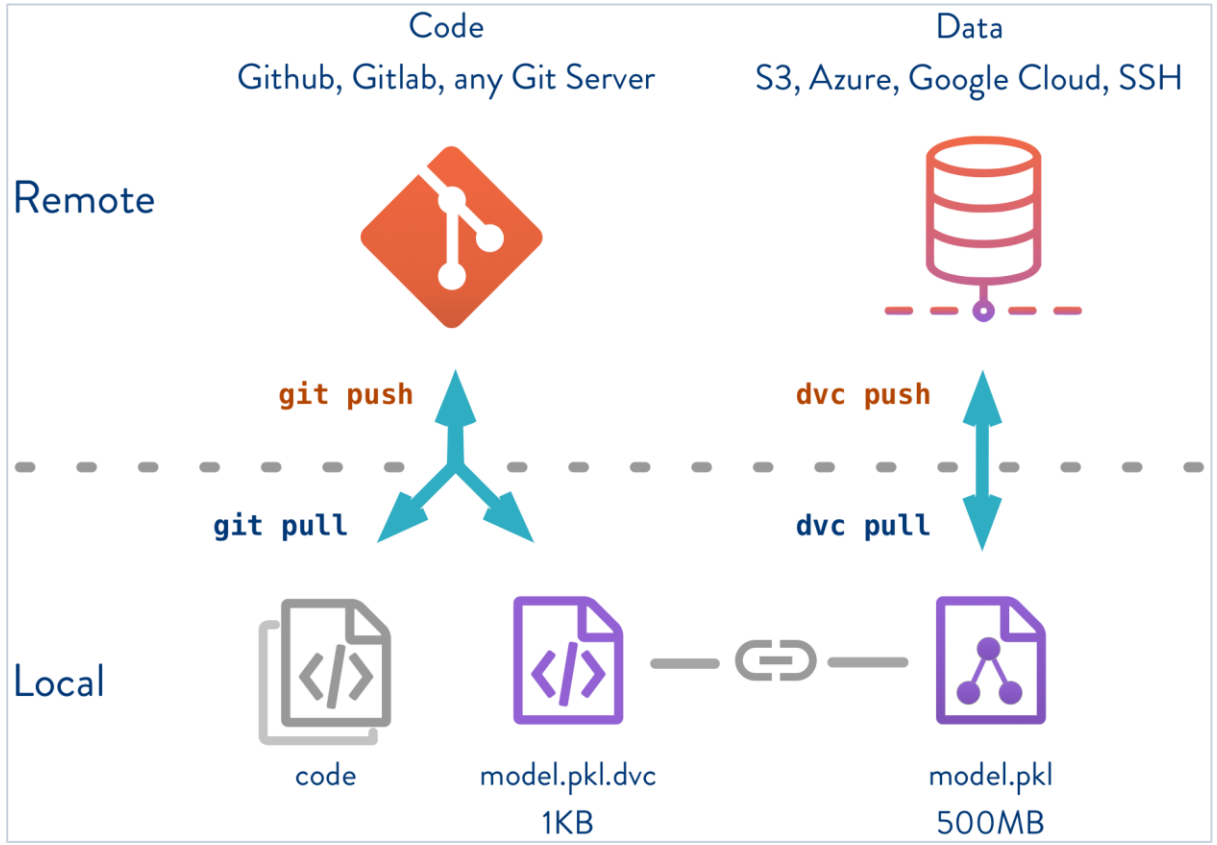
stellt den Stand her, dessen Hash in der .dvc-Datei steht

→ **32 Zeichen vergleichen statt 28 MB ➔** darum sind status, push und pull auch bei großen Daten schnell

Ausgaben: eigener Lauf mit DVC 3.67.1 am Kursdatensatz data/imagenette5, 09.11.2026

git push und dvc push: zwei Ziele

Git transportiert den Zeiger, DVC die Dateien ➔ erst beide zusammen machen einen Stand für andere nutzbar



`git push` → **Git-Server, z. B. GitHub**

imagenette5.dvc, Code, dvc.yaml

114 Byte Zeiger für den ganzen Datensatz

`dvc push` → **DVC-Remote (Share, S3 …)**

Bilder und .dir-Listen aus dem Cache

≈ 28,8 MB 3.004 Dateien – nur, was dort fehlt

⚠ **Nur git push** = Zeiger ohne Daten

→ **Nach jedem git push auch dvc push ➔** sonst zeigt der Commit auf Daten, die nur auf Ihrem Rechner liegen

Abbildung: Iterative, dvc.org-Repository (model-versioning-diagram.png), Apache-2.0 · Werte: DVC 3.67.1, 09.11.2026

Duplikate: einmal geladen, zweimal im Ordner

dvc pull überträgt jeden Inhalt einmal ➔ ob die Kopien im Arbeitsordner doppelt Platz belegen, entscheidet der Link-Typ

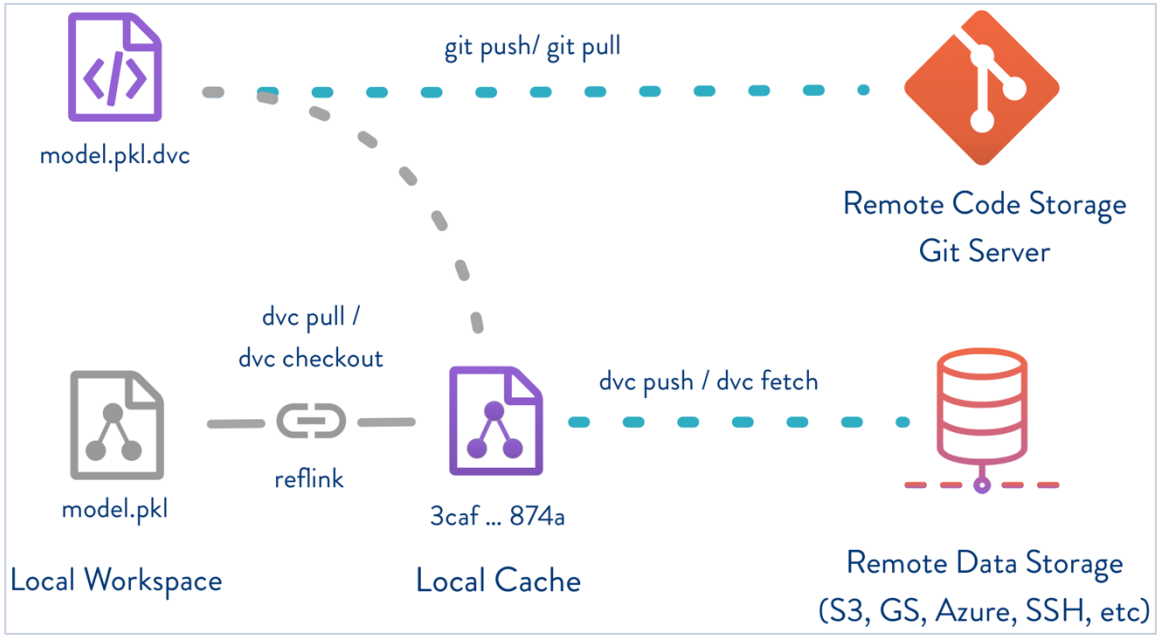
3.004 → **3.103**

Dateien geladen (eindeutige Inhalte) — Dateien im Arbeitsordner angelegt

```
$ dvc pull  3004 files fetched and 3103 files added
```

Woher die Differenz kommt

3.003	eindeutige Bilddateien	
+ 1	.dir-Liste (Inhaltsverzeichnis)	**= 3.004 geladen**
+ 100	Duplikate, lokal aus dem Cache	**= 3.103 angelegt**



Die 100 Kopien kommen nicht übers Netz: Sie entstehen beim Checkout aus denselben Cache-Dateien. Wie – Kopie oder Link – regelt cache.type (nächste Folie).

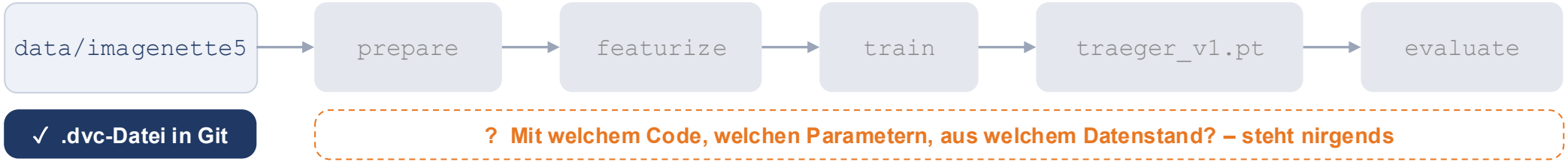
→ **Cache und Remote speichern jeden Inhalt einmal ➔** der Arbeitsordner zeigt alles, was die .dir-Liste nennt

Abbildung: Iterative, dvc.org-Repository (flow.png), Apache-2.0 · Zahlen: eigener Lauf, DVC 3.67.1

Daten versioniert – und das Modell?

Nach Aufgabe 1 kennt Git jede Version der Bilder – aber nicht, wie traeger_v1.pt daraus entstanden ist

TRÄGERMODELL NACH AUFGABE 1



1 Woraus ist traeger_v1.pt entstanden?

OHNE PIPELINE

Welche Bilder, welcher Code, welche Lernrate? Das weiß nur, wer das Notebook selbst ausgeführt hat.

2 lr_backbone geändert – was muss neu laufen?

OHNE PIPELINE

Alles von vorn – oder raten. In Lab 2 dauerten Features 21 s und Fine-Tuning 59 s (M4 Pro), auf dem Referenz-Laptop etwa doppelt so lang.

3 Ein Kollege klont das Repo – bekommt er dasselbe Modell?

OHNE PIPELINE

Nur wenn er alle Schritte in der richtigen Reihenfolge mit denselben Parametern ausführt.

→ **Versionierte Daten reichen nicht** – auch der Weg von den Daten zum Modell muss festgehalten werden

Hinzufügen oder erzeugen

Rohdaten kommen von außen und werden mit dvc add versioniert – was ein Skript erzeugt, wird Ausgabe einer Stufe



dvc add für Rohdaten

WAS	Dateien, die kein Skript erzeugt
BEISPIEL	`data/imagenette5` · heruntergeladene Gewichte
ZEIGER	`.dvc` → Hash: welcher Inhalt

Stufe in dvc.yaml für Ergebnisse

WAS	Alles, was ein Skript aus anderen Dateien erzeugt
BEISPIEL	`manifest.csv` · `features_*.pt` · `traeger_v1.pt`
ZEIGER	`dvc.lock` → Hash + Befehl, Eingaben, Parameter: wie entstanden

Warum nicht einfach `dvc add models/traeger_v1.pt` ? Dann ist das Modell gesichert, aber nicht seine Herkunft – und ans Neutrainieren müssten Sie selbst denken.

→ Faustregel: **Was ein Skript erzeugt, wird Ausgabe einer Stufe** – nicht mit dvc add versioniert

Drei Dateien, drei Rollen

dvc repro verbindet sie

`params.yaml`

≈ Prozessparameter

INHALT	Lernraten, Epochen, Batch, Seed
WER	**Sie**
WANN	bei jedem Experiment

`dvc.yaml`

≈ Arbeitsplan

INHALT	Stufen: Befehl, Eingaben, Parameter, Ausgaben
WER	**Sie**
WANN	einmal, bei neuen Stufen

`dvc.lock`

≈ Fertigungsprotokoll

INHALT	Hashes aller Ein- und Ausgaben, verwendete Parameterwerte
WER	**DVC**
WANN	nach jedem Lauf



DIE DREI VERSPRECHEN
aus „Ein Modell, viele Zutaten“

1. **Festhalten** → `dvc.lock`
2. **Feststellen** → `dvc repro` · `dvc status`
3. **Wiederherstellen** → `Zeitreise`

→ **Sie schreiben params.yaml und dvc.yaml, DVC schreibt dvc.lock ➔** alle drei gehören in Git

dvc.yaml: der Arbeitsplan

Die Pipeline als Datei

`dvc.yaml` (Ausschnitt)

```
stages:
  prepare:   …
  featurize: …
  train:
    cmd: python lab06/train.py
    deps:
      - lab06/train.py
      - lab02/src/traeger.py
      - lab06/data/manifest.csv
      - lab06/cache/features_…_train.pt
      - lab06/cache/features_…_val.pt
    params:
      - train
    outs:
      - models/traeger_v1.pt
      - models/traeger_v1.yaml
  evaluate:  …
```

`cmd` — **Was läuft?**
genau dieser Befehl

`deps` — **Wovon hängt das Ergebnis ab?**
ändert sich eine Datei, läuft die Stufe neu – auch beim eigenen Code

`params` — **Welche Stellschrauben?**
der Block train aus params.yaml

`outs` — **Was entsteht?**
kommt in den DVC-Cache, Git sieht nur den Hash

ALLE VIER STUFEN

	liest	schreibt
`prepare`	`imagenette5`	**`manifest.csv`**
`featurize`	`manifest.csv` `resnet18-….pth`	**`features_*.pt`**

`train`	`manifest.csv`
	`features_*.pt`
	`params: train`
		`traeger_v1.pt`
		`traeger_v1.yaml`

`evaluate`	`traeger_v1.pt` `traeger_v1.yaml`	**`metrics.json`**

Ausgabe einer Stufe = Eingabe der nächsten → daraus baut DVC den Graphen selbst (dvc dag). Vollständige Datei im Anhang.

→ **Was nicht in deps steht, bemerkt DVC nicht** – vergessene Abhängigkeiten sind der häufigste Fehler

dvc repro: nur rechnen, was nötig ist

Der erste Lauf rechnet alles und schreibt dvc.lock. Jeder weitere vergleicht zuerst und rechnet nur, was sich geändert hat

1. LAUF · NOCH KEIN dvc.lock

```
$ dvc repro
Running stage 'prepare':
> python lab06/prepare.py
Running stage 'featurize':
> python lab06/featurize.py
Running stage 'train':
> python lab06/train.py
Running stage 'evaluate':
> python lab06/evaluate.py
Generating lock file 'dvc.lock'
```

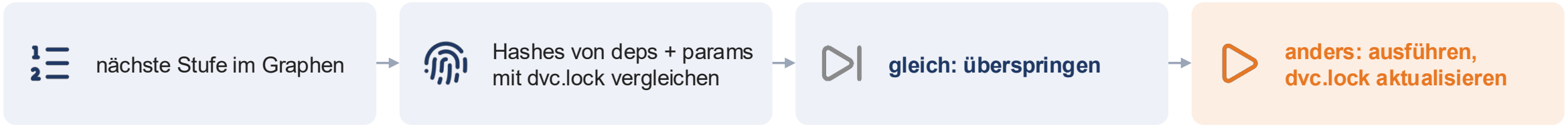
4 von 4 Stufen gerechnet → Ergebnisse im Cache, dvc.lock angelegt

2. LAUF · NICHTS GEÄNDERT

```
$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'featurize' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Data and pipelines are up to date.
```

0 von 4 Stufen gerechnet → nur Hashes verglichen

SO ENTSCHEIDET dvc repro JE STUFE



→ **Das Prinzip aus „Reproduzierbar und gecacht“ ➔** dvc repro wendet es automatisch auf jede Stufe an

dvc.lock: das Fertigungsprotokoll

Für jede Stufe: welcher Befehl, welche Eingaben, welche Parameter, welches Ergebnis

`dvc.lock` (Ausschnitt Stufe train)

```
train:
  cmd: python lab06/train.py
  deps:
  - path: lab06/train.py
    md5: 5e2f…
  - path: lab06/data/manifest.csv
    md5: 3c8a…
  - …
  params:
    params.yaml:
      train:
        finetune: {lr_backbone: 0.0001, …}
        …
  outs:
  - path: models/traeger_v1.pt
    md5: 9a41…
```

WELCHE ZUTAT STECKT IN traeger_v1.pt?

Code	`deps`	md5 von train.py, traeger.py
Daten	`deps`	md5 von manifest.csv → jedes einzelne Bild
Parameter	`params`	die tatsächlich verwendeten Werte
Ergebnis	`outs`	md5 des Modells



Zusätzlich: Run-Cache

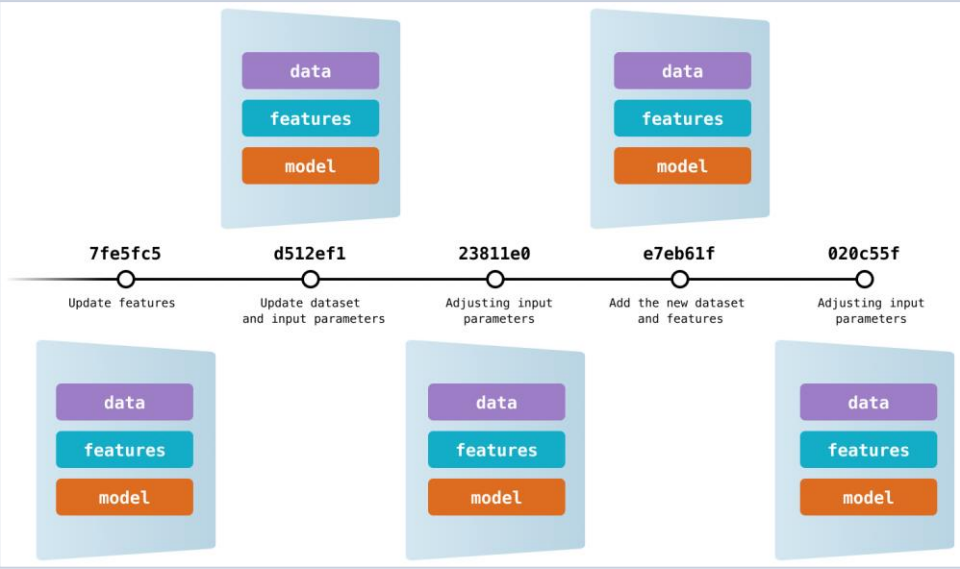
DVC merkt sich jede schon gerechnete Kombination – auch ältere. Wert zurückgesetzt → Ergebnis aus dem Cache, ohne Rechnen:

```
Stage 'train' is cached - skipping run, checking out outputs
```

→ **dvc.lock gehört in Git**: Ein Commit hält Code, Parameter, Datenhashes und Modellhash zusammen

Zeitreise: einen alten Stand wiederherstellen

Jeder Commit verweist auf genau einen Stand von Daten und Modell ➔ Git holt die Zeiger, DVC die passenden Dateien



Jeder Commit gehört zu genau einer Version von Daten, Features und Modell

```
$ git log --oneline
e72bdd8 100 Kopien
40b40fd remote
e7d84d7 Daten v1
$ git checkout 40b40fd
$ dvc checkout
M       data/imagenette5/
$ ls data/imagenette5
classes.json  train  train.txt  val  val.txt
```

→ **train_kopie/ ist weg:** Stand v1 wiederhergestellt

Vergleichen ohne Auschecken

`dvc params diff` · `dvc metrics diff`
`dvc status` – passt der Arbeitsordner zu den Zeigern?

Wiederherstellbar, wenn …

- ✓ `.dvc` committet und gepusht ist (git push)
- ✓ die Dateien im Cache oder auf dem Remote liegen (dvc push)
- ✓ niemand sie vorher mit `dvc gc` gelöscht hat

→ **Erst git checkout, dann dvc checkout ➔** auf einem Rechner ohne Cache stattdessen dvc pull

Abbildung: Iterative, dvc.org-Repository (project-versions.png), Apache-2.0 · Terminal: DVC 3.67.1, 09.11.2026

Zwischenstand: Datenversionierung mit DVC

1

Git versioniert Zeiger, DVC die Daten

.dvc-Datei: Prüfsumme, Größe, Anzahl Dateien

2

Cache nach Inhalt, Remote fürs Team

push und pull · ältere Stände nur, wenn sie gepusht wurden

3

dvc.yaml + dvc.lock = reproduzierbare Pipeline

dvc repro rechnet nur, was sich geändert hat

?

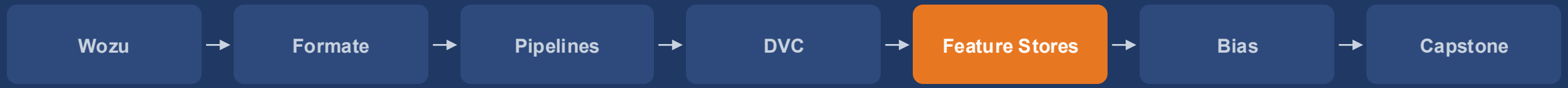
Modell v3 ist schlechter als v2. Mit welchen zwei Befehlen holen Sie v2 zurück?

Weiter: Was passiert mit Merkmalen im Betrieb? → Feature Stores

5

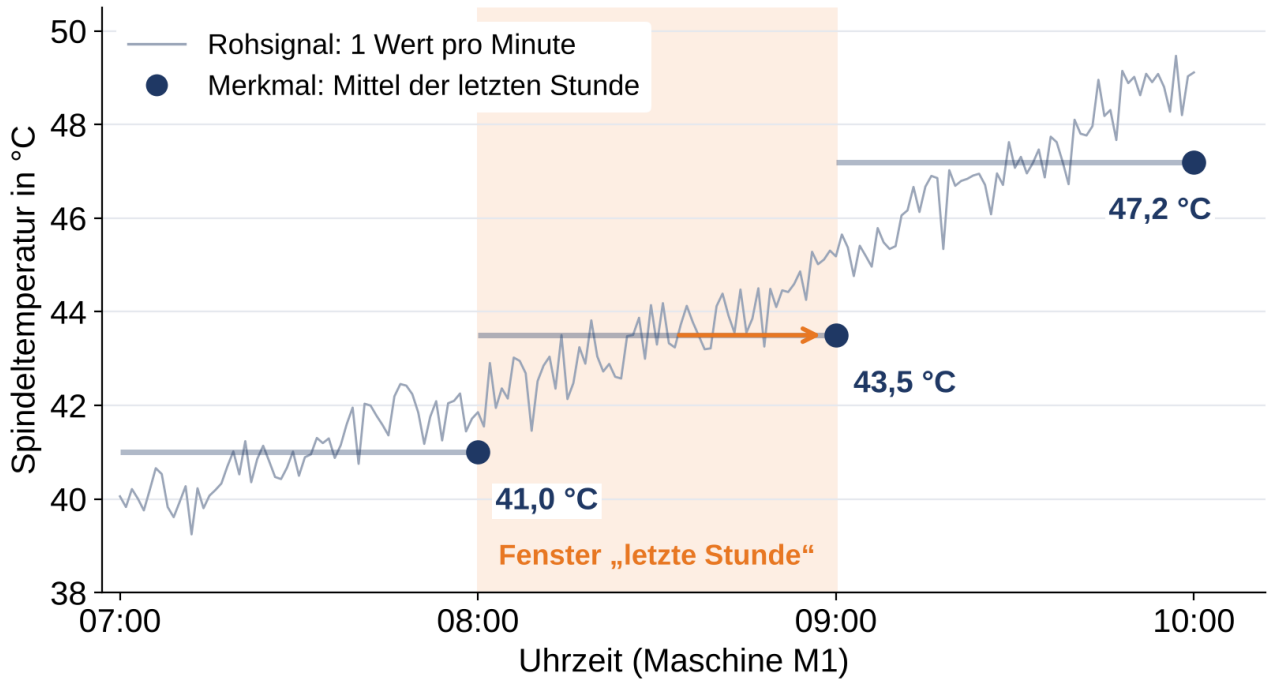
Feature Stores

Leitfrage Wie bekommt das Modell im Betrieb dieselben Merkmale wie im Training?



Was ist hier ein Merkmal?

Bei Sensor- und Tabellendaten berechnen wir die Eingaben des Modells selbst (je Maschine und Zeitpunkt)



Ein Merkmalswert hat drei Teile

Entität	Zeitpunkt	Wert
M1	**09:00**	**43,5 °C**

Name: spindel_temp_1h · Mittel der letzten 60 Minuten

Trägermodell (Bilder)

Das CNN lernt seine Merkmale selbst aus den Pixeln – kein eigener Merkmalscode

Sensor- und Tabellendaten

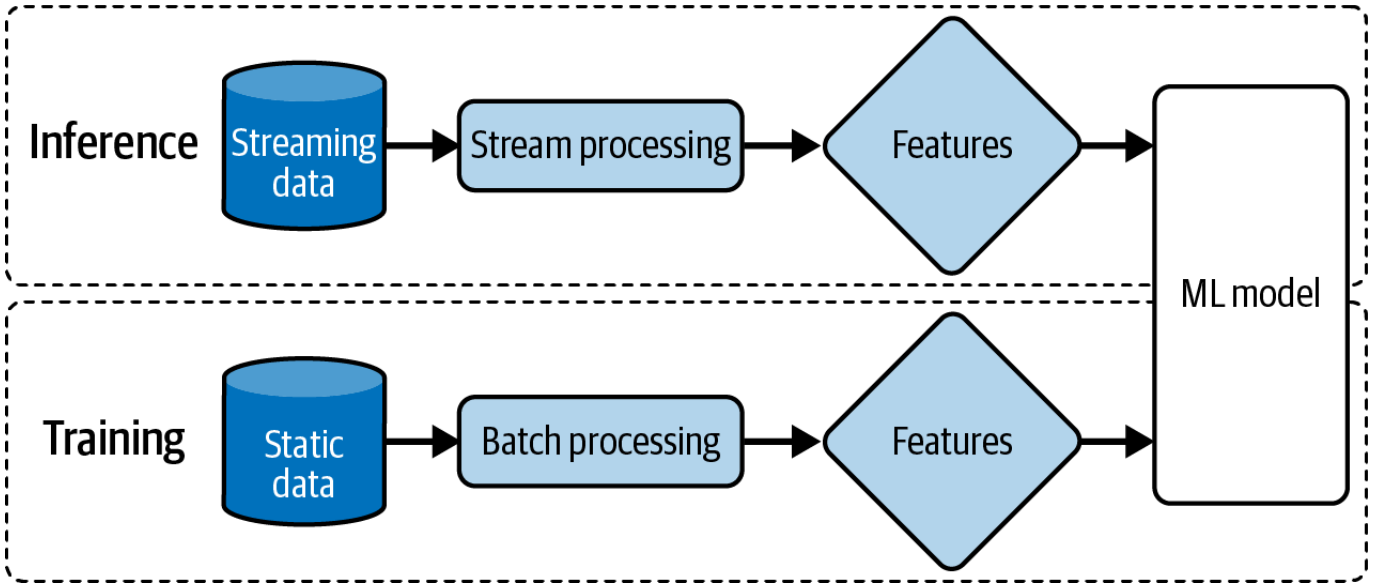
Merkmale berechnen wir selbst – mit Code, der in Training **und** Betrieb laufen muss

→ Merkmal = **berechneter Eingabewert je Entität und Zeitpunkt** – sein Code wird zweimal gebraucht: im Training und im Betrieb

Plot: simulierte Daten · Begriffe nach Feast-Dokumentation (Entity, Event Timestamp) und Huyen (2022), Kap. 5 und 7

Ein Merkmal, zwei Einsatzorte

Im Training wird das Merkmal für die Vergangenheit berechnet, im Betrieb für jetzt – oft von zwei getrennten Pipelines



Huyen, Abb. 7-7: oben Betrieb (Inference), unten Training – zwei getrennte Pipelines

	Training	Betrieb
Wann?	Vergangenheit	jetzt
Zeilen	viele auf einmal	eine je Anfrage
Daten	gespeichert	Datenstrom
Tempo	unkritisch	Millisekunden
Wer?	oft ML-Team	oft Software-Team

Gefahr 1 · Skew

Die zwei Pipelines rechnen nicht exakt gleich

Gefahr 2 · Blick in die Zukunft

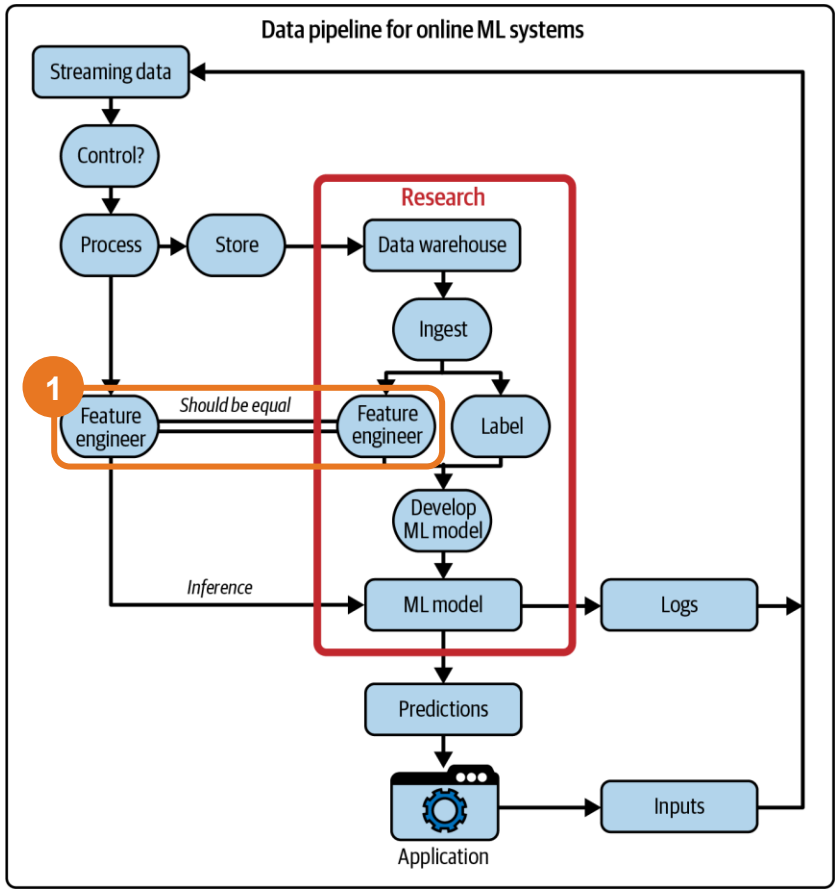
Im Training landet ein Wert, der damals noch unbekannt war

→ Beide Gefahren erzeugen **keine Fehlermeldung** – nur ein Modell, das im Betrieb schlechter ist als im Test

Abb.: Huyen, C.: Designing Machine Learning Systems, O'Reilly 2022, Abb. 7-7, S. 204 · Nutzung nur im Kurs (§ 60a UrhG)

Gefahr 1: Training-Serving-Skew

Dieselbe Merkmalsidee, zweimal programmiert – kleine Unterschiede, und das Modell sieht im Betrieb andere Zahlen



① „Should be equal“

Das Merkmal wird im Training und im Betrieb **getrennt** berechnet – und soll trotzdem exakt gleich sein.

Typische Ursachen

- fehlende Werte: weglassen, 0 oder letzter Wert?
- Zeitfenster: 60 Minuten oder 60 Messwerte?
- Rundung, Einheiten, Zeitzonen
- Python im Training, Java oder C im Betrieb

Tückisch:
kein Absturz, keine Fehlermeldung – nur schlechtere Vorhersagen

→ Gegenmittel: **Merkmalscode nur einmal schreiben** und in Training und Betrieb verwenden (Zinkevich, Regel #32)

Abb.: Huyen, C.: Designing Machine Learning Systems, O'Reilly 2022, Abb. 7-8, S. 205 · Nutzung nur im Kurs (§ 60a UrhG)

Beispiel: Gleiches Merkmal, zwei Ergebnisse

Spindeltemperatur der letzten Stunde aus sechs 10-Minuten-Werten – mit einem Sensoraussetzer

Maschine M3 · 09:00–10:00

Zeit	Temp. °C
09:10	42,0
09:20	42,8
09:30	— Aussetzer
09:40	43,6
09:50	44,0
10:00	44,6

Summe der fünf Werte: 217,0

Training · Pipeline A (pandas)

Aussetzer weglassen

217,0 / 5 = **43,4 °C**

≠

Betrieb · Pipeline B (Echtzeit-Service)

Aussetzer als 0 zählen

217,0 / 6 = **36,2 °C**

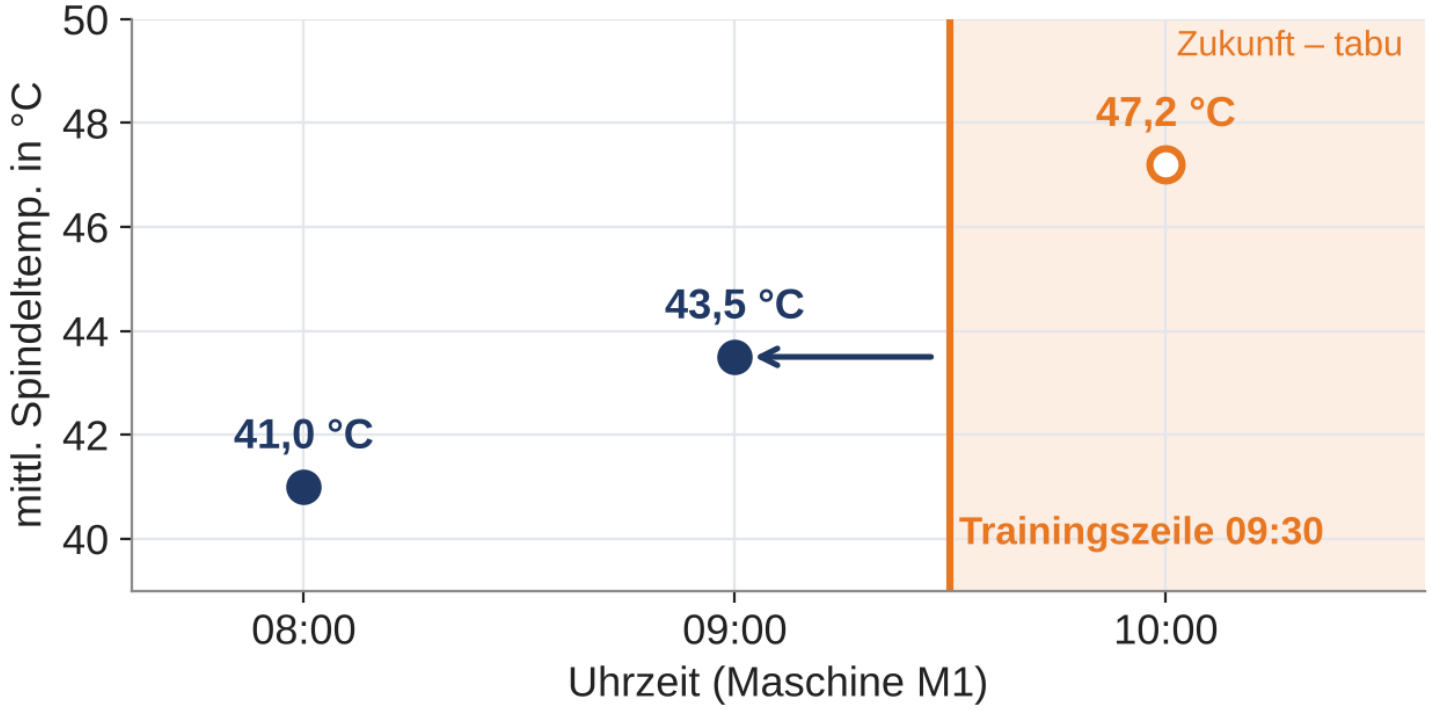
Modell gelernt: über 43 °C → Verschleißwarnung

Training: Warnung · Betrieb: **keine** – gleiche Rohdaten, keine Fehlermeldung

→ Keine Regel ist per se falsch – falsch ist, dass **Training und Betrieb verschiedene Regeln** benutzen

Gefahr 2: Blick in die Zukunft

Beim Bauen der Trainingsdaten darf jede Zeile nur Werte sehen, die zu ihrem Zeitpunkt schon bekannt waren



Naiv: „neuester Wert je Maschine“
→ 47,2 °C, gemessen erst um 10:00
→ Information aus der Zukunft = **Data Leakage** (→ Gruppenvortrag)

Richtig: Point-in-time
nur Werte ≤ 09:30 → **43,5 °C**
Feast: der Zeitstempel der Trainingszeile ist die Obergrenze

Trainingszeile: Maschine M1 · 09:30 · Label „Ausfall in 24 h?“ – welcher Temperaturwert gehört dazu?

→ **Point-in-time-korrekt** heißt: im Training so tun, als wäre es damals – nur Werte mit Zeitstempel ≤ Zeitpunkt der Zeile

Beispiel: Point-in-time-Join von Hand

Fünf Merkmalswerte, drei Trainingszeilen – welcher Wert gehört in welche Zeile?

Merkmal: Spindeltemperatur

Maschine	Zeit	Temp. °C
M1	08:00	41,0
M1	09:00	43,5
M1	10:00	47,2
M2	08:30	39,1
M2	09:45	40,3

Trainingszeilen + Ergebnis

Maschine	Zeit	Temp. °C
M1	09:30	**43,5**
M2	09:00	**39,1**
M1	10:00	**47,2**

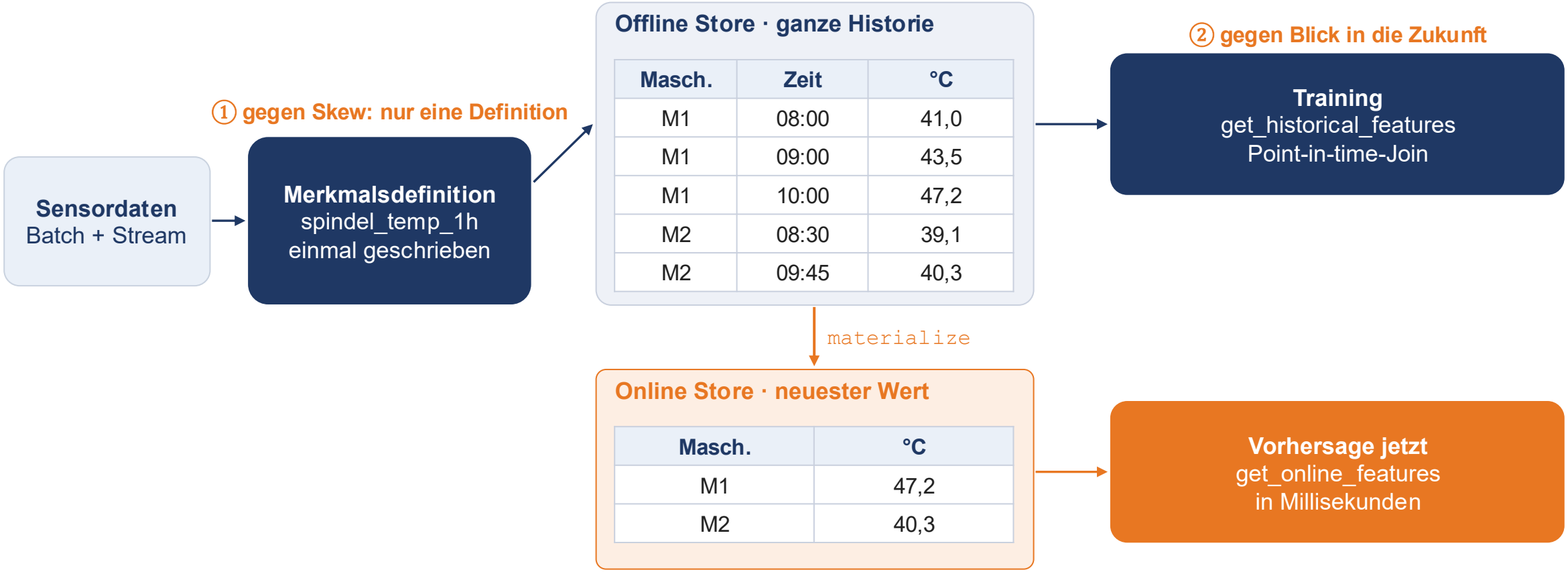
So gehen Sie vor

1. gleiche Maschine
2. nur Zeiten ≤ Zeit der Zeile
3. davon der späteste Wert

→ Genau das macht **pandas.merge_asof(…, by="maschine", direction="backward")** – Code im Anhang

Die Lösung: ein Feature Store

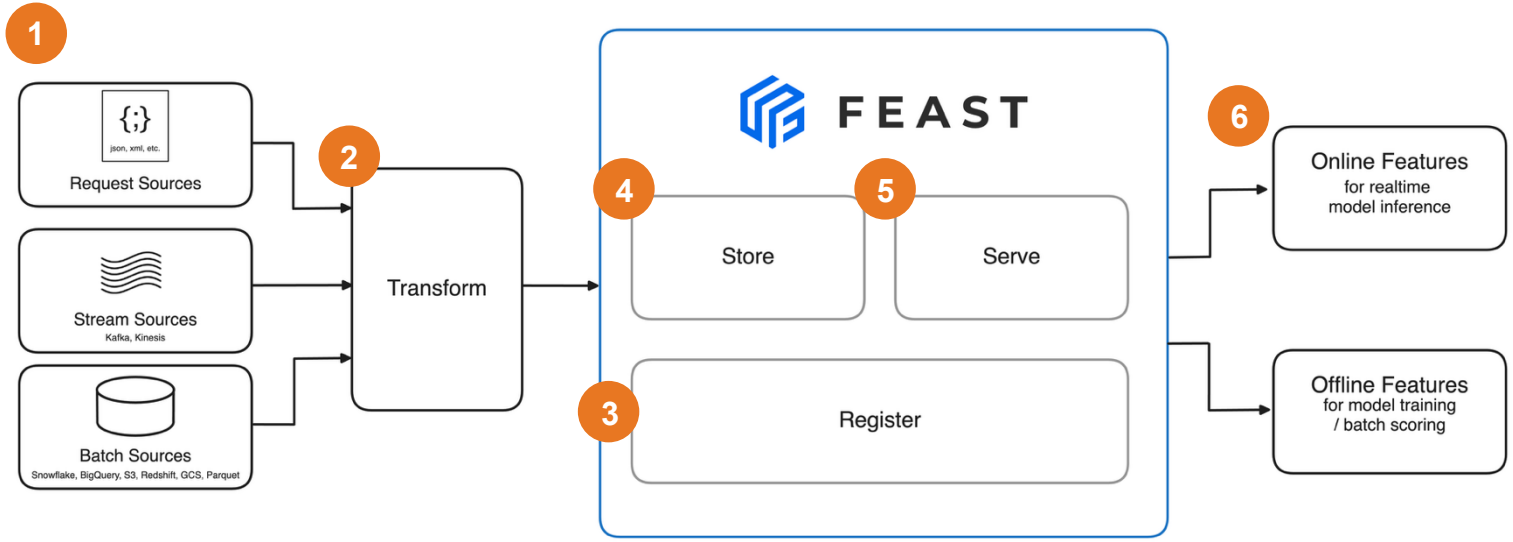
Merkmal einmal definieren + ein Speicher mit der Historie fürs Training, einer mit dem neuesten Wert für den Betrieb



→ Feature Store = **eine Merkmalsdefinition, zwei Speicher**: Historie fürs Training, neuester Wert für die Vorhersage

Feast: die Architektur gelesen

Die Abbildung aus der Feast-Dokumentation – Schritt für Schritt auf unser Schema übertragen



Wofür?

Verwaltung: Merkmale finden und teilen – Uber 2017: ≈ 10.000 Merkmale

Berechnung: teure Merkmale einmal rechnen und speichern

Konsistenz: Batch und Stream aus derselben Logik

1. **Quellen:** Anfragen, Ströme, Tabellen
2. **Transform:** Merkmale berechnen
3. **Register:** Merkmalsdefinitionen (Registry)
4. **Store:** Offline und Online Store
5. **Serve:** Werte ausliefern
6. **Ausgänge:** Vorhersage · Training

Umfrage 01/2022: ≈ 40 % von 95 Firmen nutzen einen

→ Beispiele: **Feast** (Open Source), Tecton, SageMaker, Databricks – was „Feature Store“ umfasst, ist je Anbieter verschieden

Abb.: Feast Authors, „feast_marchitecture.png“, github.com/feast-dev/feast (docs/assets), Apache License 2.0, abgerufen 04.11.2026 · Huyen (2022), Kap. 10

Brauchen wir einen Feature Store?

Erst bei vielen Merkmalen, Teams oder Echtzeit-Vorhersagen ➔ im Capstone reicht meist ein sauberer Point-in-time-Join

Trägermodell (Bilder)

Nein

- CNN lernt Merkmale selbst aus Pixeln
- keine zeitabhängigen Merkmale
- Konsistenz über traeger_v1.yaml

Capstone mit Zeitreihen

Point-in-time-Join reicht

- z. B. Lastprognose, Predictive Maintenance
- Merkmalsfunktion einmal schreiben, überall importieren
- Trainingsdaten mit merge_asof bauen

Viele Modelle und Teams

Feature Store lohnt

- Merkmale teilen – Uber 2017: ≈ 10 000
- Echtzeit-Vorhersagen mit Stream-Merkmalen
- teure Merkmale einmal rechnen

Huyen, Umfrage 01/2022: ≈ 40 % von 95 Unternehmen nutzen einen Feature Store – die Hälfte davon selbst gebaut

→ Faustregel: **Merkmalscode einmal schreiben, zeitrichtig joinen** – der Feature Store ist das Werkzeug dafür im großen Maßstab

Zwischenstand: Feature Stores

Wie bekommt das Modell im Betrieb dieselben Merkmale wie im Training?

1

Merkmale werden zweimal gebraucht

im Training für die Vergangenheit, im Betrieb für jetzt

2

Zwei Gefahren: Skew und Blick in die Zukunft

verschiedene Rechenregeln · Werte nach dem Zeitpunkt der Zeile

3

Feature Store: eine Definition, offline + online

Historie zeitrichtig fürs Training, neuester Wert für die Vorhersage

?

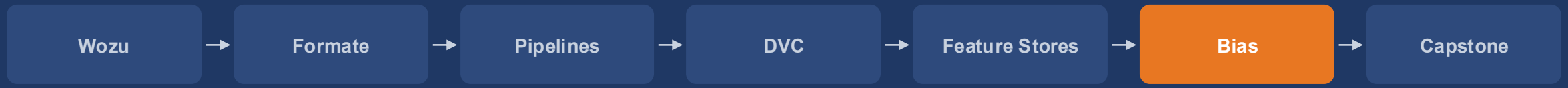
Welche Merkmale hat Ihr Capstone-Projekt – und wann ist jeder Wert bekannt?

Weiter: Stimmen unsere Daten überhaupt mit der Einsatzwelt überein? → Bias in Trainingsdaten

6

Bias in Trainingsdaten

Leitfrage Bildet der Datensatz die Einsatzwelt ab?



Wo Bias entsteht

Verzerrungen kommen beim Erheben, beim Stichprobenziehen und beim Labeln in die Daten



1 Erheben

Historische Daten enthalten menschliche Verzerrungen – das Modell schreibt sie fort

→ *Fairness: Erklärbare KI & Compliance*



2 Stichprobe

Nicht zufällig: Convenience · Snowball · Judgment · Quota – nicht repräsentativ

→ **heute**



3 Labeln

Falsche oder uneinheitliche Labels

→ *Gruppenvortrag: Label-Noise*

→ Huyen: **„Use data but don't trust it too much!“**

Sampling-Bias: drei Beispiele

Wer nimmt, was leicht verfügbar ist, bekommt nicht die Einsatzwelt



Autonomes Fahren

Daten v. a. aus Phoenix und der Bay Area – meist sonnig

2016: Waymo erweitert gezielt auf das regnerische Kirkland



natürliches Label: Sterne

Stimmungsanalyse

Rezensionen von IMDB und Amazon – natürliches Label: Sterne

nur Menschen, die online bewerten



„Name That Dataset“

Linearer Klassifikator erkennt die Quelle von 12 Datensätzen

39 % Treffer bei 8 % Zufall

→ Jeder Datensatz hat einen **Fingerabdruck** – ein Modell kann ihn statt des Objekts lernen

Quellen: Huyen (2022), Kap. 4, S. 83–84 · Torralba & Efros (2011), CVPR · Foto und Abb.: APF, https://people.csail.mit.edu/torralba/publications/datasets_cvpr11.pdf

Repräsentativität prüfen

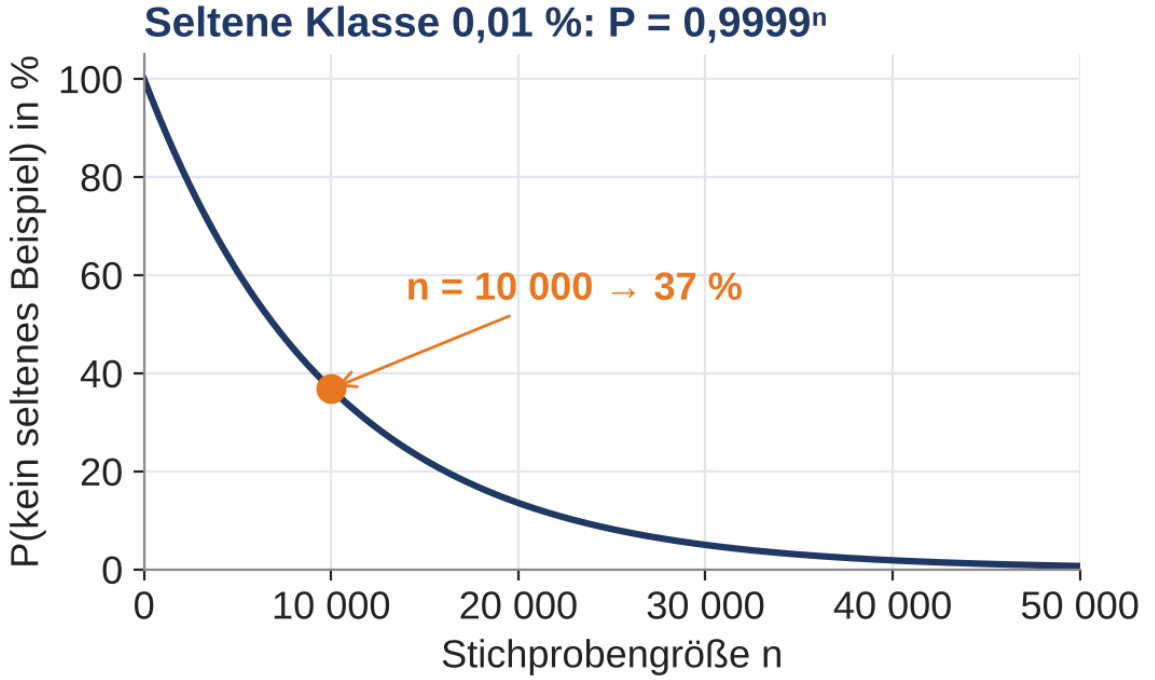
Passt die Verteilung im Datensatz zu der im Einsatz – auch innerhalb jeder Klasse?

1 Einsatzbedingungen auflisten
Licht, Kamera, Hintergrund, Standort

2 Je Bedingung zählen
Kommt sie im Datensatz vor – wie oft?

3 Stichprobe ziehen
zufällig oder geschichtet

4 Statistiken je Klasse vergleichen



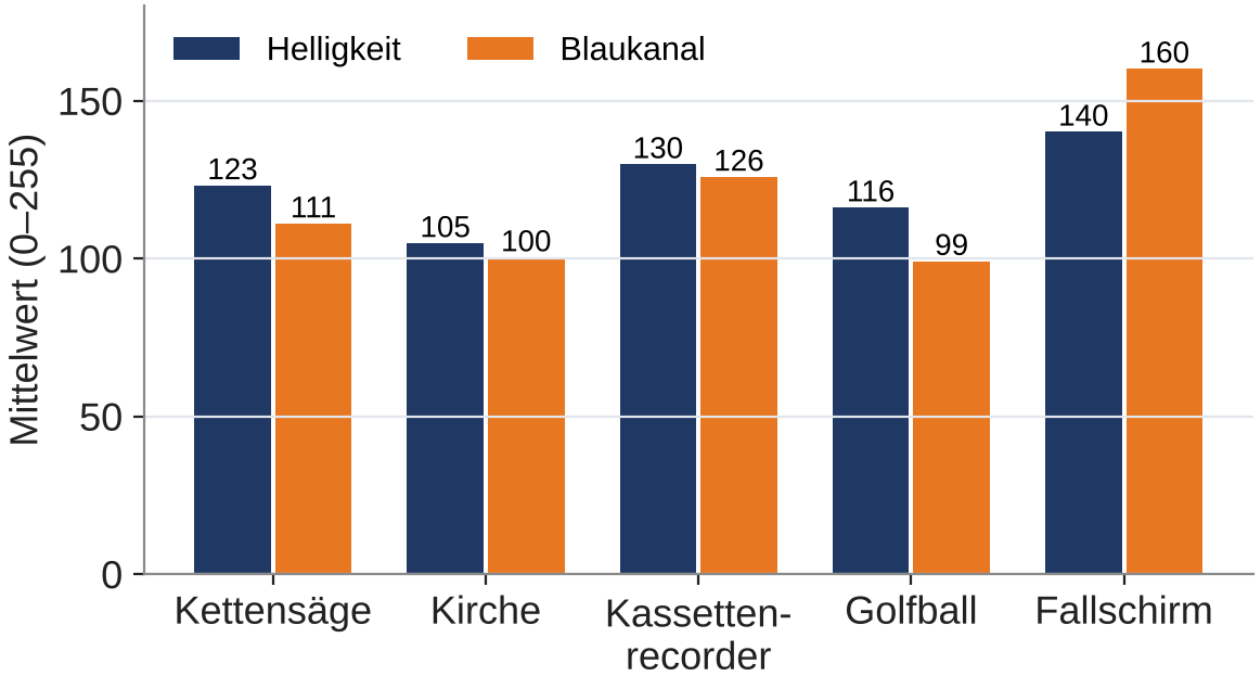
Warum Zufall nicht reicht: Geschichtet – je Gruppe 1 % – ist die seltene Gruppe sicher dabei

→ Ungleiche **Klassen** → Gruppenvortrag (Class Imbalance) · hier: ungleiche **Bedingungen innerhalb** der Klassen

nach Huyen (2022), Kap. 4, S. 84 · Rechnung ergänzt

Beispiel: Helligkeit und Farbe je Klasse

Trägermodell, 2.500 Trainingsbilder: Fallschirme sind deutlich heller und blauer als alle anderen Klassen



Fallschirm



Kirche



→ Hinweis, kein Beweis: Ob das Modell **Himmel statt Fallschirm** gelernt hat, zeigt erst die Erklärbarkeit

Messung: eigene Auswertung, data/imagenette5 (2 500 Trainingsbilder) · Bilder: Imagenette (J. Howard, fast.ai) / ImageNet – nur nicht-kommerzielle Forschung und Lehre

Zwischenstand: Bias in Trainingsdaten

Bildet der Datensatz die Einsatzwelt ab?

1

Bias entsteht beim Erheben, Stichprobenziehen, Labeln

nicht-zufällige Stichproben sind nicht repräsentativ

2

Jeder Datensatz hat einen Fingerabdruck

Name That Dataset: 39 % statt 8 % Zufall

3

Balancierte Klassen schützen nicht vor Bias

Trägermodell: Fallschirm heller und blauer – Himmel

?

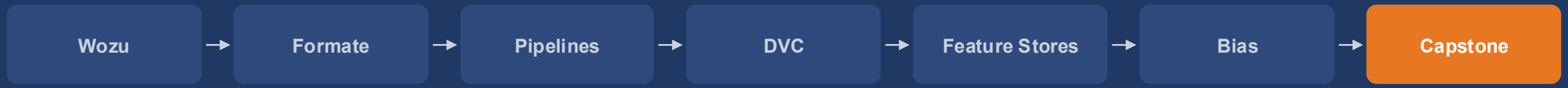
Welche Bedingung fehlt in den Daten Ihres Capstone-Themas?

Weiter: Jetzt für Ihr eigenes Projekt → Capstone-Kickoff: Datenkonzept

7

Capstone-Kickoff: Datenkonzept

Leitfrage Woher kommen Ihre Daten – und dürfen Sie sie nutzen?



Das Datenkonzept auf einer Seite

Sechs Felder, angelehnt an „Datasheets for Datasets“ – Grundlage für alles Weitere im Projekt



1 Quelle & Zugang

Woher? Wer hat die Rechte? Ist der Zugang gesichert?



2 Lizenz & Recht

Lizenz, personenbezogene Daten, DSGVO, AI-Act-Bezug



3 Umfang & Aufbau

Anzahl, Größe, Format, Merkmale, Labels, Splits



4 Qualität & Prüfung

Welche Prüfungen laufen in prepare?



5 Bias & Repräsentativität

Welche Einsatzbedingungen fehlen?



6 Versionierung & Pipeline

DVC-Remote, Stufen, wer pusht?

→ Das Datenkonzept ist **Checkpoint-Grundlage**: Zum ersten Checkpoint sind die Daten mit DVC versioniert

nach Gebru, T. et al.: Datasheets for Datasets, 2021, Communications of the ACM 64(12)

Beispiel: Datenkonzept des Trägermodells

Alle sechs Felder ausgefüllt



Imagenette: 5 Klassen

1 Quelle & Zugang

Imagenette (J. Howard, fast.ai) – 5 ImageNet-Klassen, im Kursrepo

2 Lizenz & Recht

Repo: Apache 2.0 · Bilder: ImageNet – nur nicht-kommerzielle Forschung und Lehre

3 Umfang & Aufbau

3 000 JPEG · 160×160 · 28,5 MB · 2 500/500 · je Klasse 500/100

4 Qualität & Prüfung

0 defekt · 0 Duplikate · Labels über WordNet-IDs

5 Bias & Repräsentativität

Fallschirm heller/blauer · Bilder teils aus ILSVRC-Val

6 Versionierung & Pipeline

DVC · Remote auf Share · 4 Stufen · dvc.lock in Git

Selbst ausfüllen

Jetzt, 3 Minuten, in Ihrer Gruppe:

Füllen Sie Feld 1 und 2 für Ihr Capstone-Thema aus.

Wer hält die Rechte an Ihren Daten – und an den einzelnen Inhalten?

Bilder: Imagenette (J. Howard, fast.ai) / ImageNet – nur nicht-kommerzielle Forschung und Lehre

Leitfragen und Fahrplan

Heute in der Sprechstunde: Ihr Datenkonzept – bis zum ersten Checkpoint: versionierte Daten

1. Haben Sie die Daten heute – oder wann?
2. Welche Lizenz? Darf es ins öffentliche Repo?
3. Wie wird gesplittet: Zeit, Person, Maschine?
4. Welche drei Prüfungen laufen in prepare?
5. Welche Einsatzbedingung fehlt?
6. Wo liegt das Remote, wer pusht?



Heute
Datenkonzept · Repo · Rollen



Checkpoint 1
Daten mit DVC · Baseline · erstes Modell



Checkpoint 2
Container · Drift/Logging · XAI



Abschluss
Demo · Model Card · Repo

Zusammenfassung

Vier Kernaussagen zu Daten-Engineering für ML



Das Modell ist der kleinste Teil des Systems

Daten, Prüfung, Merkmale, Betrieb – dort sitzt die technische Schuld



Ein Modell ist nur so reproduzierbar wie seine Daten

Git + DVC + dvc.lock verbinden Daten, Code, Parameter und Modell



Merkmale müssen in Training und Betrieb gleich sein

eine Definition, Point-in-time-Joins, kein Blick in die Zukunft



Repräsentativität prüft man je Bedingung, nicht nur je Klasse

Trägermodell: Fallschirm = blauer Himmel?

→ Für Ihr Projekt: **Datenkonzept + versionierte Daten** sind der erste Schritt zum Checkpoint

Bis nächste Woche

Nachbereiten, lesen, das Datenkonzept fertigstellen



1 Lab 06 nachbereiten

Pipeline komplett laufen lassen

dvc push nicht vergessen



2 Lesen

Huyen Kap. 3–4 (Pflicht)

Sculley et al. 2015 · Huyen Kap. 5 Leakage, Kap. 10 Feature Store



3 Capstone

Datenkonzept fertigstellen

Repo + DVC einrichten → Checkpoint 1



4 Ausblick

MLOps: Vom Notebook zum Produktivsystem

Trägermodell als Service: MLflow, FastAPI, Docker

A

Anhang: zum Nachlesen

Befehle · Code · Lektüre · Quellen · Abbildungen

Link-Typen: vom Cache in den Arbeitsordner

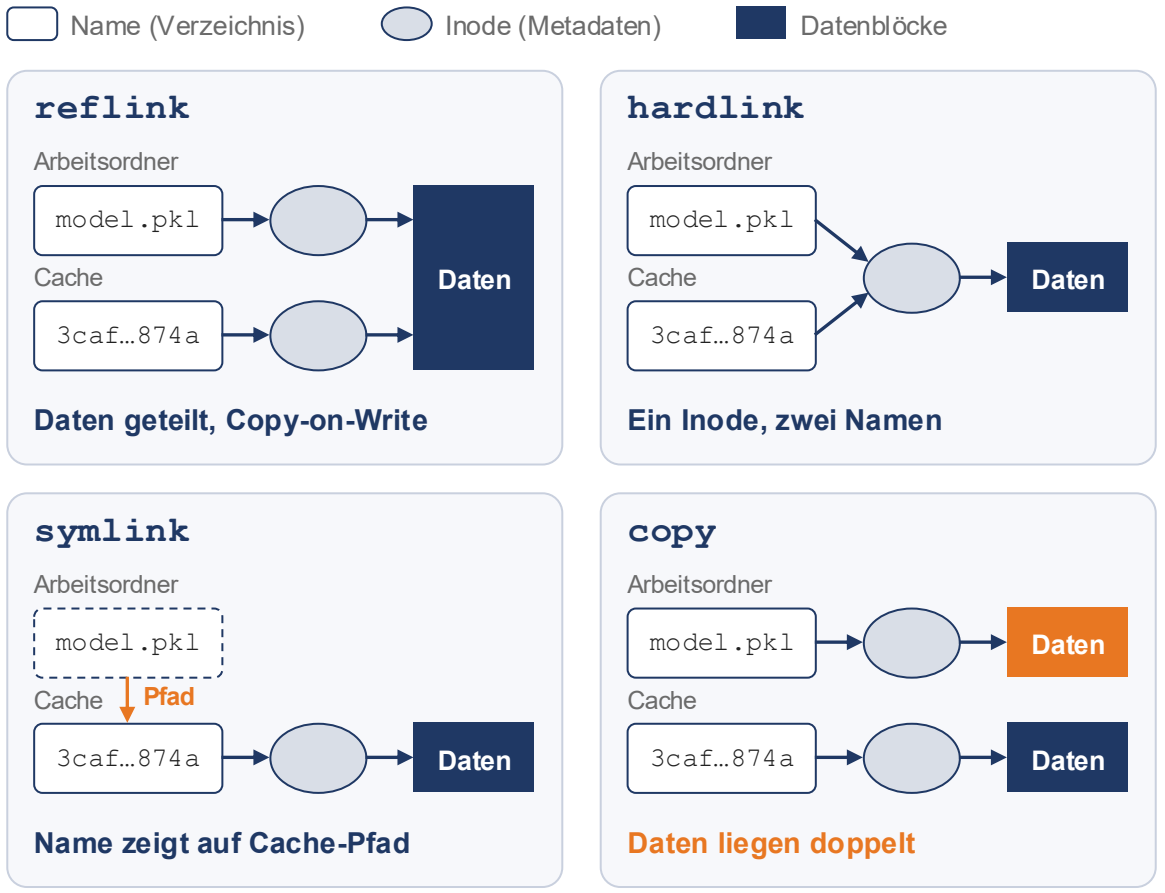
Eine Datei hat drei Ebenen: Name, Inode, Datenblöcke. cache.type legt fest, welche davon Arbeitsordner und Cache teilen

cache.type	schnell	spart Platz	editierbar
`reflink`	✓	✓	✓
`hardlink`	✓	✓	–
`symlink`	✓	✓	–
`copy`	–	–	✓

Standard: `reflink`, sonst `copy`

Unser Test (Linux, ext4): kein reflink → copy → Kopien belegen doppelt Platz. Auf macOS (APFS) greift reflink.

hardlink/symlink: Die Datei im Arbeitsordner ist der Cache-Eintrag. Vor dem Bearbeiten `dvc unprotect`, sonst wird der Cache beschädigt.



→ **Copy kostet Platz, Links kosten Vorsicht** – reflink vereint beides, wo das Dateisystem Copy-on-Write kann

Schema nach P. Brady, pixelbeat.org (CC BY-SA 3.0), angepasst · Tabelle nach doc.dvc.org

DVC-Befehlsreferenz

Alle Befehle aus Vorlesung und Lab – geprüft mit DVC 3.67.1

Befehl	Was er tut	Folie
`dvc init`	Ordner .dvc/ anlegen (einmal je Repo)	dvc add
`dvc add <pfad>`	hashen, in den Cache, .dvc-Datei + .gitignore	dvc add
`dvc status`	Arbeitsordner vs. .dvc-Dateien und dvc.lock	repro
`dvc remote add -d <name> <url>`	Remote anlegen und als Standard setzen	Remote
`dvc push [--all-commits]`	Cache → Remote (Standard: aktueller Stand)	Remote
`dvc pull`	Remote → Cache → Arbeitsordner	Remote
`dvc repro [-f]`	Pipeline ausführen, nur Geändertes	repro
`dvc dag`	Pipeline als Graph anzeigen	Lab 2
`dvc checkout`	Daten passend zu den Zeigern herstellen	Zeitreise
`dvc params diff · dvc metrics show | diff`	Parameter und Metriken vergleichen	Zeitreise

Code: dvc.yaml und params.yaml vollständig

Die Pipeline des Trägermodells – getestet mit DVC 3.67.1, Stufen-Skripte im Lab-Paket

```
stages:
  prepare:
    cmd: python lab06/prepare.py
    deps: [lab06/prepare.py, data/imagenette5]
    outs: [lab06/data/manifest.csv]
    metrics: [lab06/reports/datencheck.json: {cache: false}]
  featurize:
    cmd: python lab06/featurize.py
    deps: [lab06/featurize.py, lab02/src/traeger.py, lab06/data/manifest.csv,
           data/models/resnet18-f37072fd.pth]
    outs: [lab06/cache/features_resnet18_160_train.pt,
           lab06/cache/features_resnet18_160_val.pt]
  train:
    cmd: python lab06/train.py
    deps: [lab06/train.py, lab02/src/traeger.py, lab06/data/manifest.csv,
           lab06/cache/features_resnet18_160_train.pt,
           lab06/cache/features_resnet18_160_val.pt]
    params: [train]
    outs: [models/traeger_v1.pt, models/traeger_v1.yaml]
  evaluate:
    cmd: python lab06/evaluate.py
    deps: [lab06/evaluate.py, lab02/src/traeger.py, models/traeger_v1.pt,
           models/traeger_v1.yaml]
    metrics: [lab06/reports/metrics.json: {cache: false}]
```

params.yaml

```
train:
  seed: 0
  head: {lr: 0.01,
    epochs: 10, batch: 64}
  finetune: {
    lr_head: 0.001,
    lr_backbone: 0.0001,
    epochs: 2, batch: 32}
```

Code: lab06/prepare.py

Prüft jede Datei, schreibt Manifest mit Prüfsummen und Datencheck – lauffähig, < 1 s

```
import collections, hashlib, json
from pathlib import Path
from PIL import Image

ROOT, OUT = Path("data/imagenette5"), Path("lab06")
rows, defekt, groesse = [], 0, collections.Counter()
for split in ("train", "val"):
    for line in (ROOT / f"{split}.txt").read_text().splitlines():
        rel, y = line.split("\t")
        try:
            with Image.open(ROOT / rel) as im:
                groesse[f"{im.width}x{im.height}"] += 1
                im.verify()                      # Datei lesbar und vollständig?
        except Exception:
            defekt += 1
        md5 = hashlib.md5((ROOT / rel).read_bytes()).hexdigest()
        rows.append(f"{split},{rel},{y},{md5}")
(OUT / "data").mkdir(parents=True, exist_ok=True)
(OUT / "data/manifest.csv").write_text("split,pfad,label,md5\n" + "\n".join(rows) + "\n")
dup = len(rows) - len({r.rsplit(",", 1)[1] for r in rows})
(OUT / "reports").mkdir(exist_ok=True)
json.dump({"n": len(rows), "defekt": defekt, "duplikate": dup, "groessen": groesse},
          open(OUT / "reports/datencheck.json", "w"), indent=1)
```

Code: lab06/bias_check.py

Mittlere Helligkeit und Blaukanal je Klasse – lauffähig, ≈ 4 s für 2 500 Bilder

```
import json
import numpy as np, pandas as pd
from pathlib import Path
from PIL import Image

ROOT = Path("data/imagenette5")
klassen = [c["name"] for c in json.loads((ROOT / "classes.json").read_text())]
zeilen = []
for line in (ROOT / "train.txt").read_text().splitlines():
    rel, y = line.split("\t")
    im = Image.open(ROOT / rel).convert("RGB")
    hell = np.asarray(im.convert("L"), float).mean()   # 0,299 R + 0,587 G + 0,114 B
    blau = np.asarray(im, float)[..., 2].mean()
    zeilen.append((klassen[int(y)], hell, blau))
df = pd.DataFrame(zeilen, columns=["klasse", "hell", "blau"])
print(df.klasse.value_counts())
print(df.groupby("klasse", sort=False).agg(["mean", "std"]).round(1))
```

Code: Point-in-time-Join mit pandas

merge_asof sucht je Zeile den letzten Merkmalswert ≤ Zeitpunkt – das Beispiel von der Folie

```
import pandas as pd

t = lambda s: pd.Timestamp(f"2026-11-12 {s}")
merkmal = pd.DataFrame({"maschine": ["M1", "M1", "M1", "M2", "M2"],
                        "zeit": [t("08:00"), t("09:00"), t("10:00"), t("08:30"), t("09:45")],
                        "spindel_temp": [41.0, 43.5, 47.2, 39.1, 40.3]})
training = pd.DataFrame({"maschine": ["M1", "M2", "M1"],
                         "zeit": [t("09:30"), t("09:00"), t("10:00")]})
pit = pd.merge_asof(training.sort_values("zeit"), merkmal.sort_values("zeit"),
                    on="zeit", by="maschine",
                    direction="backward")      # letzter Wert <= Zeitpunkt der Zeile
print(pit)
```

Ausgabe

```
  maschine                zeit  spindel_temp
0       M2 2026-11-12 09:00:00          39.1
1       M1 2026-11-12 09:30:00          43.5
2       M1 2026-11-12 10:00:00          47.2
```